

EEE 402 (January 2026)

Artificial Intelligence and Machine Learning Laboratory

Final Project Report

Section: G2 Group: 01

EEG-VJEPa: Self-Supervised JEPa for EEG Classification

Course Instructors:

Tanvir Hossain, Lecturer

Bangladesh University of Engineering and Technology

Anindya Bhattacharjee, Adjunct Lecturer

Bangladesh University of Engineering and Technology

Signature of Instructor: _____

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: _____

Full Name: Ahmed Mashfiqur Rahman

Student ID: 2106114

Signature: _____

Full Name: Soumya Das Prottay

Student ID: 2106116

Signature: _____

Full Name: Maruf Hasan Mahi

Student ID: 2106121

Signature: _____

Full Name:

Student ID:

Contents

Acronyms and Notation.....	ii
1 Abstract.....	1
2 Introduction.....	1
3 Design.....	3
3.1 Problem Formulation.....	3
3.2 Design Method.....	4
3.3 Data and Preprocessing Pipeline.....	17
3.4 Core Source Code and Reproducibility.....	20
4 Implementation.....	22
4.1 Description.....	22
5 Design Analysis and Evaluation.....	23
5.1 Novelty.....	23
5.2 Design Considerations.....	23
5.3 Investigations.....	24
5.4 Limitations of Tools.....	40
5.5 Impact Assessment.....	41
5.6 Sustainability Evaluation.....	41
5.7 Ethical Issues.....	41
6 Reflection on Individual and Team work.....	42
6.1 Individual Contribution of Each Member.....	42
6.2 Mode of TeamWork.....	42
6.3 Diversity Statement of Team.....	42
6.4 Log Book of Project Implementation.....	42
7 Communication to External Stakeholders.....	43
7.1 Executive Summary.....	43
7.2 Reproducing Our Results.....	44
8 Project Management and Cost Analysis.....	44
8.1 Computational Resources Used.....	44
8.2 Timeline of Project Implementation.....	45
9 Future Work.....	45
10 References.....	45
11 Appendix: Reproducibility Records.....	47
11.1 Environment and Checkpoint Provenance.....	47
11.2 Stroke Cohort: Pre-registration, Post-hoc Input and Fold-local Check.....	47
11.3 Experiment 5 Corrected Statistics.....	48
11.4 Reproducibility Files.....	48
11.5 Checks Not Run.....	49
11.6 Pre-training Configuration and Commands.....	50
11.7 Statistical Procedures.....	50
11.8 Stroke Spectral-Matching Filter.....	51
11.9 Data Used by Each Experiment.....	52

Acronyms and Notation

Acronyms and abbreviations

Acronym	Meaning
10-NN	10-nearest-neighbour agreement: the share of each point's 10 nearest neighbours in a 2-D map that carry its own label
Acc	Accuracy
AdamW	Adam optimiser with decoupled weight decay
AI	Artificial intelligence
AUC-PR	Area under the precision–recall curve
AUROC	Area under the receiver operating characteristic (ROC) curve
B _{Acc}	Balanced accuracy, $\frac{1}{2} \cdot (\text{TPR} + \text{TNR})$
BatchNorm	Batch normalisation (BatchNorm1d: PyTorch's 1-D batch-normalisation layer)
bf16	bfloat16, a 16-bit floating-point number format
BSI / pdBSI	Brain Symmetry Index / its pairwise-derived variant
BYOL	Bootstrap Your Own Latent, a momentum-encoder self-supervised method
C, FEI+C	Earlier working names for GTJ and FEI+GTJ, still used as labels in Figures 12, 17 and 19
CBraMod	Criss-Cross Brain Foundation Model, a large pre-trained EEG model
CI	Confidence interval
CPU / GPU	Central / graphics processing unit
cuDNN	NVIDIA's CUDA Deep Neural Network library
CV	Cross-validation
DAR / DTABR	Delta/alpha power ratio / (delta + theta)/(alpha + beta) power ratio
DUA / NEDC	Data Use Agreement / Neural Engineering Data Consortium, the TUH corpus's maintainers
EDF	European Data Format, a standard file format for EEG recordings
EEG	Electroencephalography
EEG-VJEPa / VJEPa	The published EEG adaptation of V-JEPa that this report reproduces; “VJEPa” labels its released checkpoint in some tables and figures
EMA	Exponential moving average
F1	Harmonic mean of precision and sensitivity
FEI	Frequency-masked Embedding Inference, our frequency-masking encoder
FEI+GTJ	The FEI and GTJ embeddings concatenated and read by one probe
FFT / rFFT / STFT	Fast Fourier transform / its real-input form / short-time Fourier transform
GB	Gigabyte
GELU / ReLU	Gaussian error linear unit / rectified linear unit (activation functions)
GRU	Gated recurrent unit
GTJ	GRU-based Time-JEPa, our spectral-dynamics encoder
I-JEPa / V-JEPa	Image / video JEPa
JEPa	Joint-Embedding Predictive Architecture
L1 / L2 / MSE	Absolute-difference (L1) distance / Euclidean (L2) length / mean squared error
LaBraM	Large Brain Model, a large pre-trained EEG model
LogReg	Logistic regression
LOSO	Leave-one-subject-out cross-validation
LRU	Linear Recurrent Unit, a long-memory recurrent layer
MLP	Multi-layer perceptron

Acronym	Meaning
NMT	The NMT Scalp EEG Dataset, an open clinical EEG corpus from Pakistan
PSD	Power spectral density
rand-	Prefix for a random-initialisation control: the same architecture with untrained weights
s0–s4	The five TUAB training subsets, 546 recordings each
sd	Standard deviation
Sens / Spec	Sensitivity (TPR) / specificity (TNR)
t-SNE / UMAP	t-distributed stochastic neighbour embedding / uniform manifold approximation and projection, two 2-D visualisation methods
TP / FP / TN / FN	True positives / false positives / true negatives / false negatives
TPR / FPR / TNR	True-positive / false-positive / true-negative rate
TUAB / TUH	TUH Abnormal EEG Corpus / Temple University Hospital
ViT / ViT-M / ViT-B	Vision Transformer / the two released EEG-VJEPa checkpoints, with 384- and 768-number tokens

Notation for FEI's objective (§3.2, Code Snippet 1)

Symbol	Meaning
x	One EEG window: 19 channels $\times$ L samples
L	Window length in samples: 1,000 (5 s at 200 Hz) for FEI pre-training; 4,000 (20 s) for GTJ's pre-training and for every FEI and GTJ probe-time embedding in Experiments 5, 6 and 8
r	FEI's frequency-removal rate for one window, drawn from $U(0, 0.7)$
M	Binary mask over the window's 501 rFFT bins (1 = removed), shared by all 19 channels
x'	The frequency-masked copy of x , $\text{irfft}(\text{rfft}(x) \cdot (1 - M))$
u, u'	Online embedding of x and EMA-target embedding of x' , after projection to h numbers
m	The mask M embedded as a prompt, $\text{mask_enc}(M)$
$\hat{u}, \hat{m}$	FEI's two predictions, of u' and of m
$n(\cdot)$	L2 normalisation, $n(v) = v/\ v\ $
$\text{sg}(\cdot)$	Stop-gradient: passes its input forward but blocks gradients flowing back through it
h	Width of the projected embeddings, 128 (the encoder output itself has 256 numbers)
$\theta_{\text{online}}, \theta_{\text{target}}, a$	Online and EMA-target weights, and the EMA momentum $a = 0.996$
ℓ	FEI's per-window pre-training loss
$\pm$	Mean $\pm$ standard deviation, over folds, training subsets or seeds as each table states

1 Abstract

EEG-VJEPa is a published self-supervised model that adapts video-style JEPa pre-training to EEG, reporting strong results on the TUH Abnormal EEG Corpus (TUAB) [1]. We set out to reproduce it and then build on it. The reproduction failed. The released, pre-trained checkpoints are **representation-collapsed**: on two completely different datasets, almost every recording we fed in came out as nearly the same output vector, so a classifier built on top of it does no better than one built on random, untrained weights. The collapse is not explained by any preprocessing variation we tested and appears to be a property of the released weights. We followed the paper's own pipeline as closely as its text and code allow (43 of 48 files byte-identical to the authors' public repository) and still could not reach their reported accuracy.

We built our own two-branch alternative in response: **FEI** (Frequency-masked Embedding Inference), a 1-D convolutional encoder pre-trained to predict, from an intact EEG window, how a slowly-updated copy of itself embeds the same window with random frequency bins removed, and **GTJ** (GRU-based Time-JEPa), a spectrogram encoder pre-trained by predicting the future course of a recording's spectrum from its past. Fused together (FEI+GTJ), our original pre-training run reaches an AUROC of 0.865 ± 0.024 on five-fold cross-validation on the NMT dataset (2,417 recordings). Averaged over that run and two independently seeded re-runs it scores 0.861, ahead of a classical hand-crafted band-power baseline scored on the identical folds (0.786 ± 0.009 ; better in all 5 folds, Holm-corrected $p = 0.029$). An untrained twin of the same fused architecture, averaged over five random seeds, already reaches 0.834, though, so most of that margin comes from the architecture: pre-training adds +0.027 AUROC (0.861 against 0.834), better in all 5 folds but not statistically significant after correction. On a completely different hospital's data (TUAB), FEI pre-trained on NMT alone, with no TUAB recording in its pre-training (only the linear classifier on top is trained on labelled TUAB data), scored above a random-weight twin by +0.048 (95% CI [0.015, 0.108], $p = 0.0055$) AUROC, averaged over five independently seeded pre-training runs scored on the same test set, with runs, random-init seeds and test recordings resampled together (the original run alone: +0.059, $p = 0.002$). GTJ did not survive that NMT-only transfer (0.840 vs 0.863 for its random twin), so we also re-pre-trained both encoders on NMT plus TUAB's unlabelled training recordings: the fused model then scored close to the released ViT-M checkpoint under our linear probe (0.862 vs 0.838, difference not significant) and beat band power (0.757), but was not significantly better than its own random-weight twin (0.861) in any of three pre-training runs. We then took the NMT-only encoders to a third, unrelated task: telling stroke patients apart from healthy controls in a small resting-state EEG cohort. There, after a spectrum-matching step we designed post hoc on this same cohort (the input fixed in advance gave 0.833), FEI+GTJ scored close to the clinical Brain Symmetry Index (BSI) baseline (0.926 vs 0.907 AUROC, leave-one-subject-out: a model is validated on one subject it never saw any data from, repeated once per subject). With 15 subjects that difference is far too uncertain to show either superiority or equivalence. Two seeded re-runs of its NMT pre-training scored 0.963 and 0.981 on the same 15 subjects: a check on the pre-training seed, not a larger sample.

Every pre-trained encoder in this report is checked against a **random-initialisation control**: the same architecture, untrained weights, same pipeline (one disclosed exception: the released ViT-B in Experiment 1). That habit is what caught the released model's collapse in the first place, and what later caught our own GTJ quietly collapsing under a spectrum shift between datasets, which we then repaired, post hoc and on that same cohort, with a label-free spectral-matching step. FEI over-flags healthy children; GTJ is fragile under distribution shift. Those findings limit where either encoder can be trusted.

2 Introduction

Reading an EEG automatically (deciding whether a recording looks "normal" or "abnormal", or telling a stroke patient's brain activity apart from a healthy control's) runs into the same wall most clinical machine learning does: labelled EEG is expensive. It takes a trained neurologist to read and annotate every recording, so the datasets available for supervised learning stay small next to what made deep learning work for images or text. The standard fix is self-supervised pre-training. Train a big encoder on a task that needs no labels, then re-use it on the small labelled task you actually care about.

Meta AI introduced I-JEPa in 2023 [2] and V-JEPa in 2024 [3]. Instead of reconstructing missing pixels, the model predicts the *embedding* of a missing region from the embedding of the visible region, using a slowly-updated ("momentum") copy of itself as the target. The masking drives the learning. Mask enough regions, and the only way the context encoder keeps succeeding at this prediction is to learn how the signal's parts fit

together. Raw pixel values (or raw voltage samples, for EEG) are full of detail an encoder would waste capacity reproducing exactly. Predicting in embedding space lets it skip that and keep what is actually predictable. That tends to be what a downstream classifier needs anyway. It also sidesteps a known failure mode of predictive self-supervision: representation collapse, where the easiest way to shrink the loss is to output the same vector for everything. Keeping the two networks from sharing weights directly is what stops that from happening.

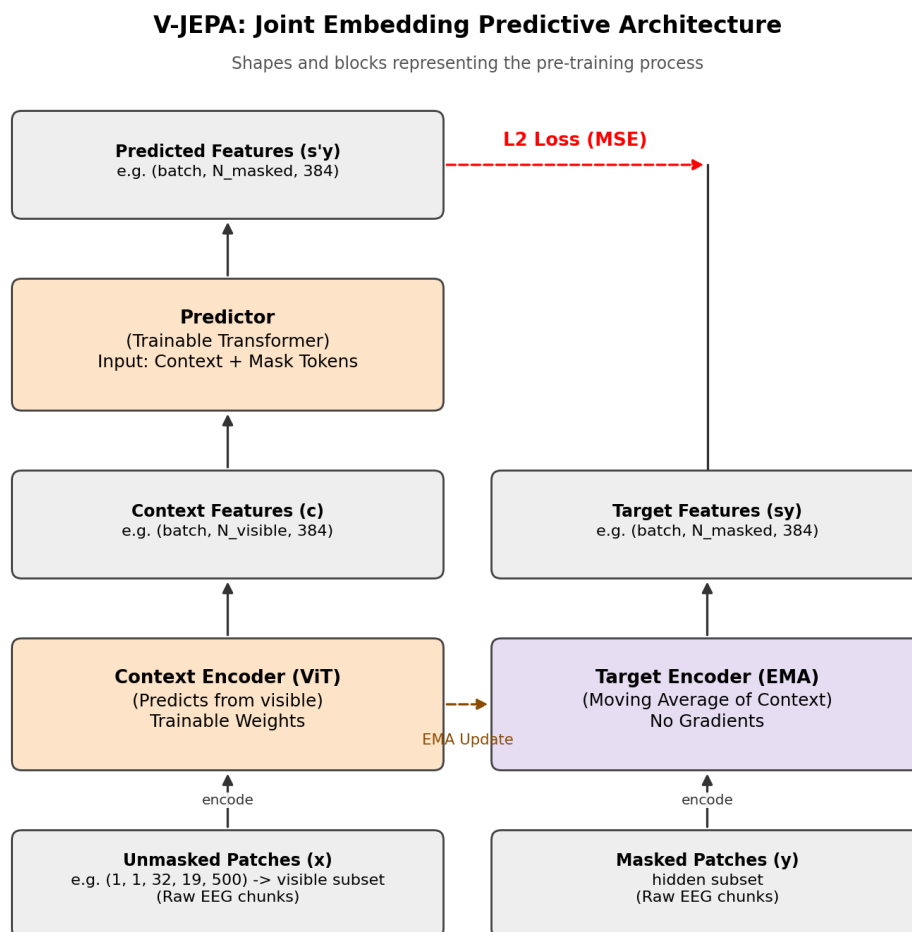


Figure 1: The JEPA idea: predict the target encoder's embedding of a masked/future region from the context encoder's embedding, never the raw pixels [2, 3].

EEG-VJEPA (Hojjati et al., 2025, arXiv:2507.03633) [1] carries this recipe over to EEG by treating a multi-channel recording as a short silent video: channels stand in for spatial rows, time windows for video frames. The authors report their model, pre-trained this way and evaluated on TUAB, reaches an AUROC around 0.877–0.879, ahead of prior published baselines. They also released their pre-trained checkpoints and code, so picking this paper felt like the safe option: reproduce a recent, credible-looking result, understand why it works, then push it further on a harder or different task. The “safe” part turned out to be the wrong assumption. We first tried, as faithfully as we could, to reproduce the paper's own TUAB result with its own released weights and its own evaluation code. That did **not** succeed, and the reason turned out to be diagnosable in its own right: the released checkpoints are collapsed. We used that as the starting point for the rest of the project (“Act 0”, in our own naming, since everything else the course builds on sits downstream of it): we designed our own pre-trained encoder, FEI+GTJ, validated it on the dataset it was trained on (NMT), tested it on TUAB, a different hospital's data, both with encoders pre-trained on NMT only and after adding TUAB's unlabelled recordings to pre-training, and finally applied it to a third, out-of-domain task (separating stroke patients from healthy controls) where a well-established clinical baseline (the Brain Symmetry Index) let us check whether

our features add anything over classical signal processing.

3 Design

3.1 Problem Formulation

3.1.1 Identification of Scope

Our original course proposal, submitted in week 5, had two parts. Part 1: reproduce EEG-VJEPa's published TUAB result using the authors' released checkpoints, and explain any gap. Part 2: apply the same frozen encoders, together with a classical clinical feature (the Brain Symmetry Index), to a healthy-vs-stroke task on an independent cohort. A model that only works on the one benchmark it was tuned for is not worth much.

A third item was in the original proposal too: pre-training our own copy of EEG-VJEPa from scratch. We dropped it partway through the term. By then it was clear that working out *why* the released model collapses, and building a replacement that doesn't, would use our remaining weeks and our one GPU better than a from-scratch run would.

We call this scope “**Act 0**” internally: reproduction, diagnosis, first working replacement. It does not cover the motor-imagery work the team has since started on a different task; that is out of scope here and shows up only in Future Work.

3.1.2 Literature Review

The **JEPa family** anchors one corner of this: I-JEPa (Assran et al., 2023) [2] and its video extension V-JEPa [3] predict embeddings instead of pixels, building on BYOL's momentum-encoder idea (Grill et al., 2020) [4]: a slowly-updated target network, no negative pairs, and in principle no collapse. Experiment 3 below shows that safeguard can still be switched off elsewhere in a training config.

A much larger body of work is **EEG foundation models** proper. Besides EEG-VJEPa, two bigger EEG transformers came out around the same time: CBraMod (Wang et al., 2025) [5] and LaBraM (Jiang et al., 2024) [6], both pre-trained on tens of thousands of hours of pooled EEG. CBraMod reports about 0.83 balanced accuracy / 0.92 AUROC on TUAB; LaBraM, about 0.81 / 0.90. These set the current bar, and gave us a sense of how a large, well-resourced EEG model is normally reported.

NMT (Khan et al., 2022) [7] is the open, South-Asian-population source we pre-train on and cross-validate against. TUAB (López et al., 2015) [8] is the U.S. benchmark the EEG-VJEPa paper targets. The Brain Symmetry Index (van Putten & Tavy, 2004; van Putten, 2007 [9, 10]) is a decades-old hand-crafted feature that tracks left-right power asymmetry and is known to track stroke severity. It's a real yardstick for the stroke experiments; we didn't pick it because it's weak. One more reference we almost missed: a methodological preprint on stress-testing EEG foundation models with negative controls (arXiv:2607.24519 [11]) surfaced partway through the project, well after we had already started comparing every pre-trained model against a random-init twin on the same pipeline. It turned out to be arguing for the exact habit we'd already fallen into: welcome confirmation, but not something that changed our approach.

3.1.3 Formulation of Problem

Concretely, the project chases one question after another, each answer setting up the next. Do the released EEG-VJEPa checkpoints reproduce their own paper's TUAB result, under a faithful re-implementation of the authors' own evaluation protocol, and if not, why not? If they don't, can we build a self-supervised EEG encoder, trained entirely within our own compute budget, that actually beats a hand-crafted classical baseline on the data it was trained on, and actually transfers that benefit, not just its architecture, to a recording from a different hospital? And if that holds up, does the same encoder, frozen and unchanged, add anything on a third, structurally different task (stroke vs. healthy), measured directly against an established clinical feature?

3.1.4 Analysis

Each question above maps to a specific check, spelled out in full in §5.3.1: question (1) against a random, untrained control (exactly what caught the collapse in Experiment 1); question (2) with paired statistics rather than a bare point estimate, since our folds, cohorts, and seeds are all small enough for a raw accuracy number

to just be noise; question (3) against the Brain Symmetry Index, a bar already established by classical neuroscience rather than one we picked ourselves.

3.2 Design Method

The released EEG-VJEPa [1] treats a 19-channel, 5-second EEG window as a single-frame “video clip”: channels take the place of one spatial axis, short time-slices take the place of the other, and a Vision Transformer (ViT-M or ViT-B) patch-embeds this grid exactly as it would a video frame. During pre-training, V-JEPa masks out spatiotemporal blocks of this grid and trains a context encoder to predict a momentum target encoder's embedding of the masked blocks.

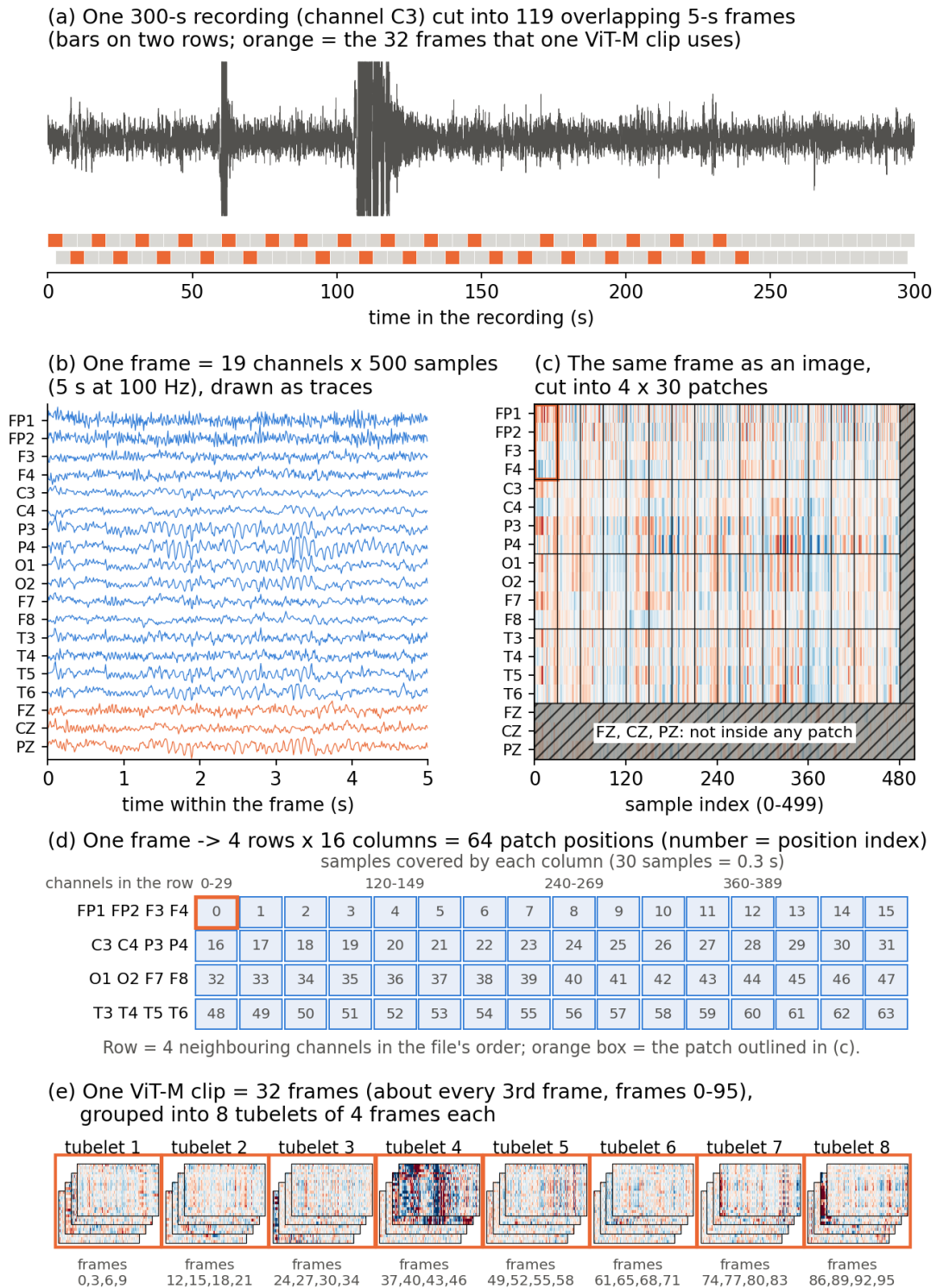


Figure 2: How the released EEG-VJEPA reframes a multi-channel EEG window as a single video frame for its Vision Transformer encoder [1].

Concretely: each 19×500 frame is cut into patches of $4 \text{ channels} \times 30 \text{ samples}$, and groups of 4 consecutive frames are joined into one “tubelet” (the same trick ordinary V-JEPA [3] uses to turn a short video clip into a manageable number of tokens). This yields 512 tokens per clip, each described by 384 numbers for ViT-M, which then pass through 12 transformer blocks: self-attention with 6 heads per block, so every token can draw on every other token, followed by a small $384 \rightarrow 1,536 \rightarrow 384$ MLP inside each block (ViT-B instead uses 768-number tokens, 12 heads, and 16-frame clips).

Released ViT-M (4x30x4) encoder: shapes recorded on one real TUAB clip

Shapes are (batch, ...). Batch = 1 clip here.

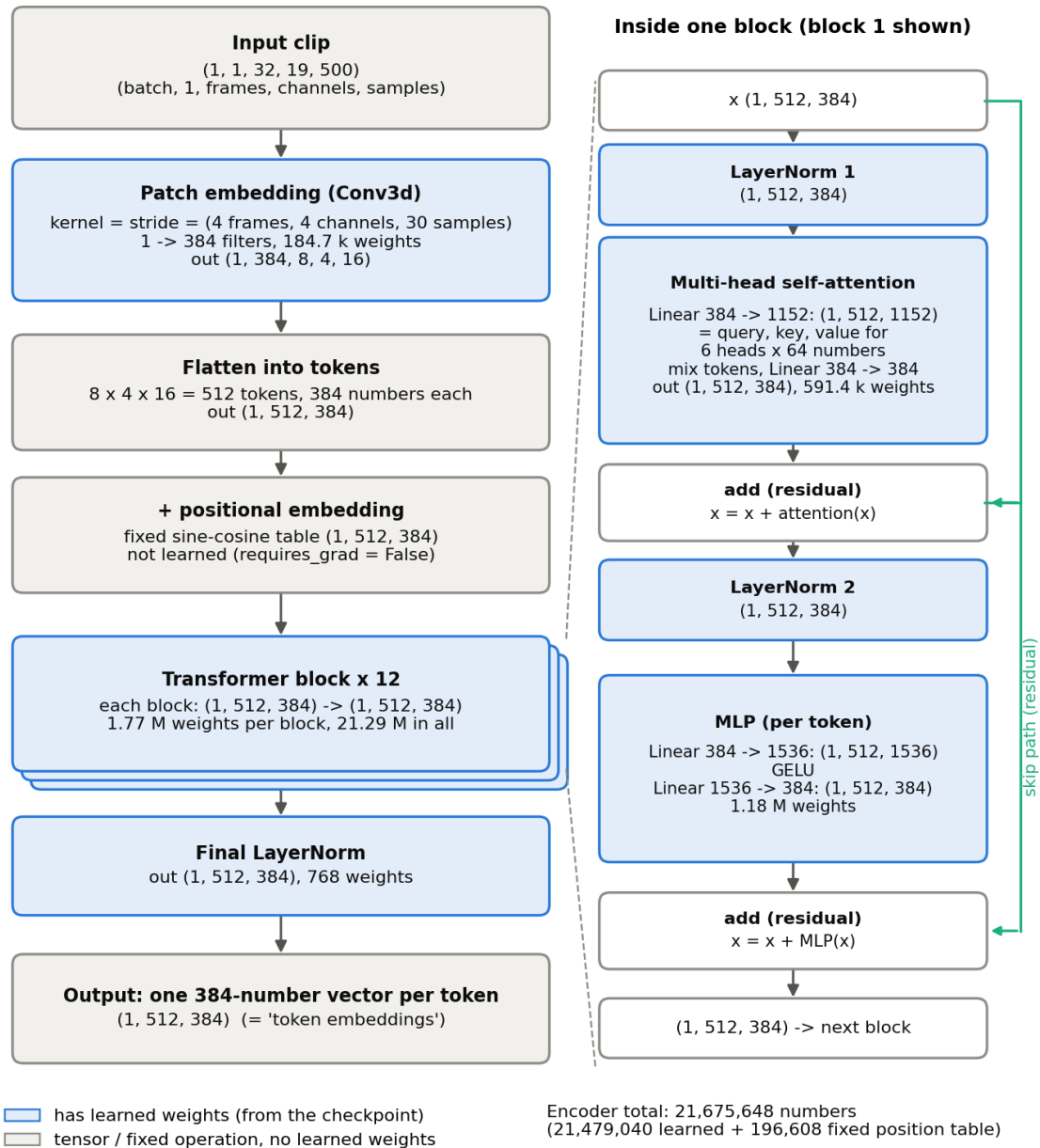


Figure 3: EEG-VJEPa's Vision Transformer — patch and tubelet sizes, the resulting 512-token $\times$ 384-number grid, and the stack of self-attention blocks that process it [1].

The three-part JEPa structure that this report refers back to throughout is, in the released model's form: a **context encoder** that only ever sees the *visible* tokens of a clip; a **target encoder** — an exponential-moving-average (EMA), never directly trained copy of the context encoder — that sees the *entire*, unmasked clip, whose outputs at the *masked* positions become the prediction targets; and a small **predictor** network that takes the context encoder's tokens plus learned “mask tokens” marking the hidden positions, and tries to predict the target encoder's output there. Only the *embedding* is predicted, never the raw signal. The released checkpoint's training loss is the mean L1 distance between predicted and target embeddings, a choice we return to directly in Experiment 3. Both of our own encoders below keep the same three parts (online encoder, EMA target encoder, predictor) but change what each encoder sees: in FEI the online encoder sees the full window and the target encoder a frequency-masked copy of it; in GTJ the online encoder sees a window's past and the target encoder its future.

Concretely, the target encoder's weights are never touched by gradient descent at all: after every training step

they are nudged a small amount towards the context (“online”) encoder's current weights [1], $\theta_{\text{target}} \leftarrow a \cdot \theta_{\text{target}} + (1 - a) \cdot \theta_{\text{online}}$, with momentum $a = 0.996$ throughout this report (so each update moves the target only 0.4% of the way towards the online encoder). This keeps the prediction target almost static from one step to the next, which is what lets the predictor chase a slowly-moving goal instead of one that shifts as fast as the network producing it.

Real masks from the authors' mask generator (ViT-M, $8 \times 4 \times 16 = 512$ tokens)

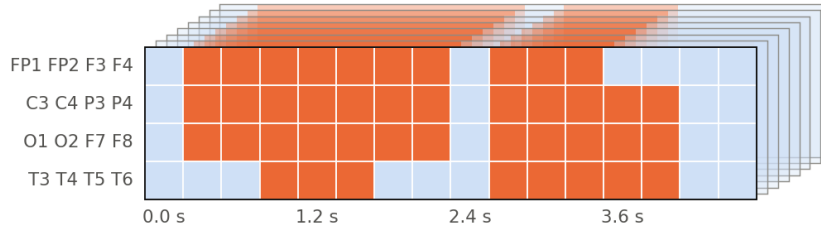
Blue = context: the encoder sees these tokens.

Orange = hidden: the predictor must guess the target encoder's vectors for these tokens.

(a) Short-range mask: 8 blocks, each asked to cover 15 % of the 4×16 slice

-> each block is 3-4 rows x 2-3 columns and spans all 8 tubelet times (the stack behind the front slice).

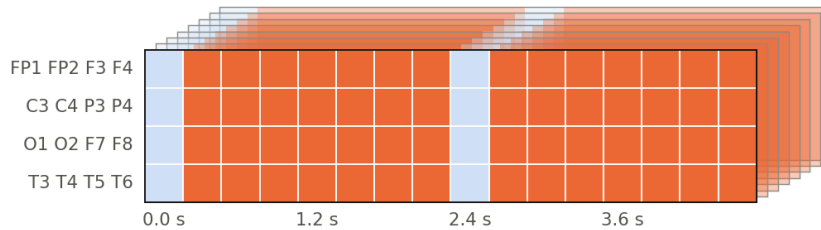
This step: context = 176 tokens, hidden = 336 of 512 (66 %).



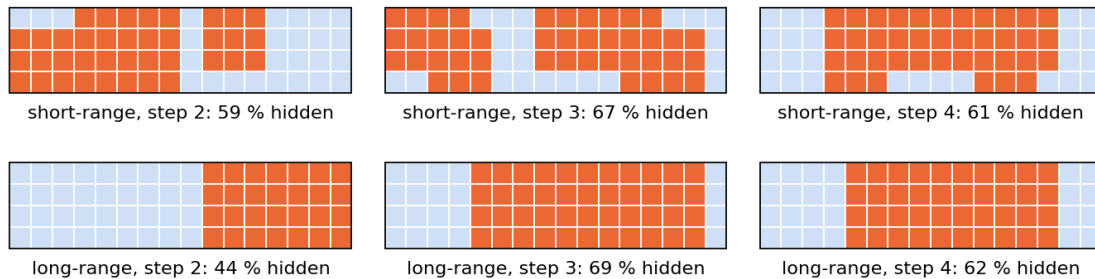
(b) Long-range mask: 2 blocks, each asked to cover 70 % of the 4×16 slice

-> each block is all 4 rows x 5-8 columns and spans all 8 tubelet times (the stack behind the front slice).

This step: context = 64 tokens, hidden = 448 of 512 (88 %).



(c) Three more training steps: a new random mask every step (front slice only).



Average over 500 steps: 68 % hidden (short-range), 60 % (long-range). Blocks can overlap, so the share varies.

Figure 4: Spatiotemporal block masking — the context encoder only ever sees the unshaded tokens; the target encoder sees every token and supplies the prediction targets at the shaded, masked positions [1].

Once we had established (Experiments 1–4, Results section) that this released checkpoint is collapsed and cannot be rescued by fine-tuning, we designed our own encoder pair from scratch, built around two principles the released model's failure pointed us towards: mask in the **frequency domain**, where pathological EEG slowing shows up directly as a spectral shift that a time-domain patch mask can miss; and keep the architecture **small and matched to the dataset size** (NMT has only a few thousand recordings, far fewer than the millions of video clips typical JEPA training uses), so that the encoder is not so over-parameterised that the easiest solution to its pretext task is to ignore the input.

FEI is a 1-D convolutional encoder. Its pretext task removes a random fraction — anywhere from 0% to 70% of rFFT frequency bins, redrawn independently for every window ($U(0, 0.7)$ in code/fei_pretrain.py) — of the signal's frequency content (via an FFT mask) and asks an online encoder to predict, in embedding space, what a slowly-updated momentum copy of itself sees when given the *masked* window, plus a small auxiliary term that predicts which bands were removed. Both prediction targets are L2-normalised before the loss is

taken (a BYOL-style trick, equivalent to a bounded $2-2\cdot\cos(\text{pred}, \text{target})$ loss), which rules out one trivial solution: an un-normalised MSE loss would let the network cheat by shrinking its output scale to zero.

```
class FEI(nn.Module):
    def __init__(self, n_freq, d=256, h=128, in_ch=NCH):
        super().__init__()
        self.enc, self.proj = Encoder(d, ch=in_ch), nn.Linear(d, h)      # online
        self.enc_t, self.proj_t = Encoder(d, ch=in_ch), nn.Linear(d, h)  # momentum target
        self._sync(); [p.requires_grad_(False) for p in self._tgt_params()]
        self.mask_enc = nn.Linear(n_freq, h, bias=False)                # mask prompt -> embedding space
        self.z1, self.z2 = mlp(h), mlp(h)                               # predictors

    def forward(self, x, xp, M):
        e = self.enc(x); u = self.proj(e)                                # online, original window
        with torch.no_grad():
            up = self.proj_t(self.enc_t(xp))                            # momentum, masked window
            m = self.mask_enc(M)
            uhat = self.z1(u + m.detach())                               # predict masked embedding
            mhat = self.z2(u.detach() - up)                             # predict which bands were
removed
        nrm = lambda a: F.normalize(a, dim=-1)                            # BYOL-style: bounded loss, no
scale collapse
        l_embed = F.mse_loss(nrm(uhat), nrm(up), reduction="none").mean(-1)
        return l_embed.mean() + F.mse_loss(nrm(mhat), nrm(m)), e, l_embed.detach()

def freq_mask(x, bias=0.0, b1=0.0, b2=0.7):
    """Zero a random U(b1,b2) fraction of rFFT bins (shared across channels)."""
    Xf = torch.fft.rfft(x, dim=-1)                                     # (B,19,n) complex
    ratio = torch.empty(x.shape[0], 1, device=x.device).uniform_(b1, b2)
    M = (torch.rand(x.shape[0], Xf.shape[-1], device=x.device) < ratio).float()
    xp = torch.fft.irfft(Xf * (1 - M).unsqueeze(1), n=x.shape[-1], dim=-1)
    return xp.float(), M
```

Code Snippet 1: FEI's momentum-encoder JEPA loss and its frequency-domain masking function (code/fei_pretrain.py, simplified — the full file also implements an adaptive, difficulty-weighted masking variant, `--mask_bias``, whose only use in this report is in the fused scores of GTJ's window-length sweep later in this section).

Written out once, for one window x ($19 \text{ channels} \times L \text{ samples}$, $L = 1,000$ in pre-training): `freq_mask` draws a removal rate $r \sim U(0, 0.7)$, removes each of the 501 rFFT bins independently with probability r (the same bins on every channel, recorded in a binary mask M), and inverts the FFT to give the masked copy $x' = \text{irfft}(\text{rfft}(x) \cdot (1 - M))$. The online path embeds the original window, $u = \text{proj}(\text{enc}(x))$; the target path embeds the masked copy without gradient, $u' = \text{proj}_t(\text{enc}_t(x'))$; and the mask itself is embedded as a prompt, $m = \text{mask_enc}(M)$. The two predictors are $\hat{u} = z_1(u + \text{sg}(m))$ and $\hat{m} = z_2(\text{sg}(u) - u')$, where $\text{sg}(\cdot)$ is stop-gradient; with $n(v) = v/\|v\|$ and $h = 128$, the loss, averaged over the batch, is

$$\ell = (1/h) \cdot \ln(\hat{u}) - n(u')\|^2 + (1/h) \cdot \ln(\hat{m}) - n(m)\|^2$$

The first term trains the encoder, its projection and z_1 ; the second trains only z_2 and the mask-prompt encoder, because $\text{sg}(\cdot)$ stops its gradient from reaching u . The target weights (`enc_t`, `proj_t`) are never trained by gradient: after each step they move towards the online weights by the EMA rule above, with $a = 0.996$.

The figure below lays out that whole step at once, for a concrete batch of 4 windows: the frequency masking, the online and target encoders, both predictor heads, and where the EMA update happens.

One FEI pre-training step (real shapes, batch of 4 windows)

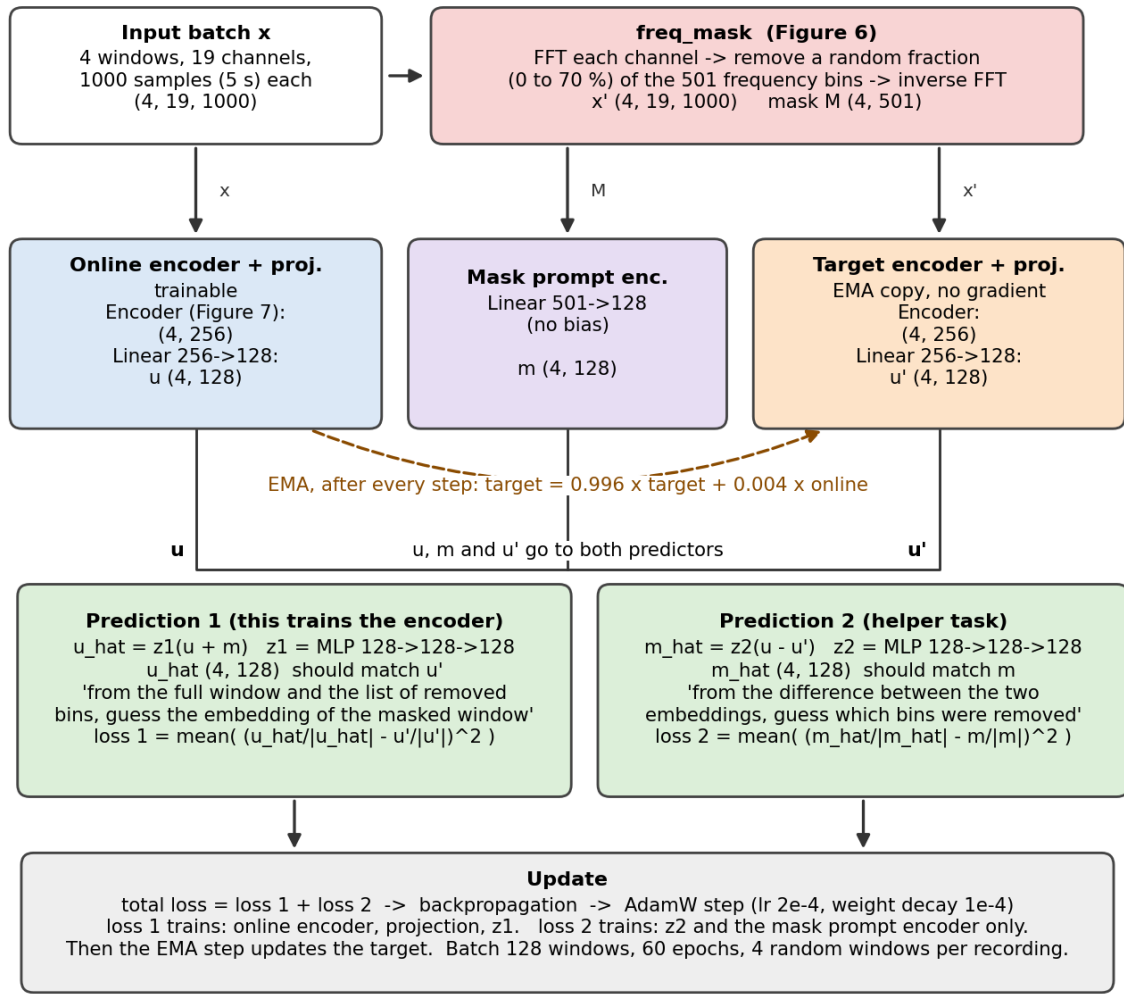


Figure 5: One FEI pre-training step for a batch of 4 windows — frequency masking produces the masked copy x' and the mask M ; the online encoder embeds the original window x while the EMA target encoder embeds the masked copy; prediction 1 recovers the target's masked-window embedding from the online embedding plus the mask prompt, prediction 2 recovers the mask prompt from the difference between the two embeddings; the summed loss updates the online branch and the mask-prompt encoder by AdamW and the target branch by EMA.

The next figure shows `freq_mask` acting on one real NMT channel's window rather than in the abstract: which rFFT bins actually get zeroed in one draw, and what that removal does to the time-domain signal.

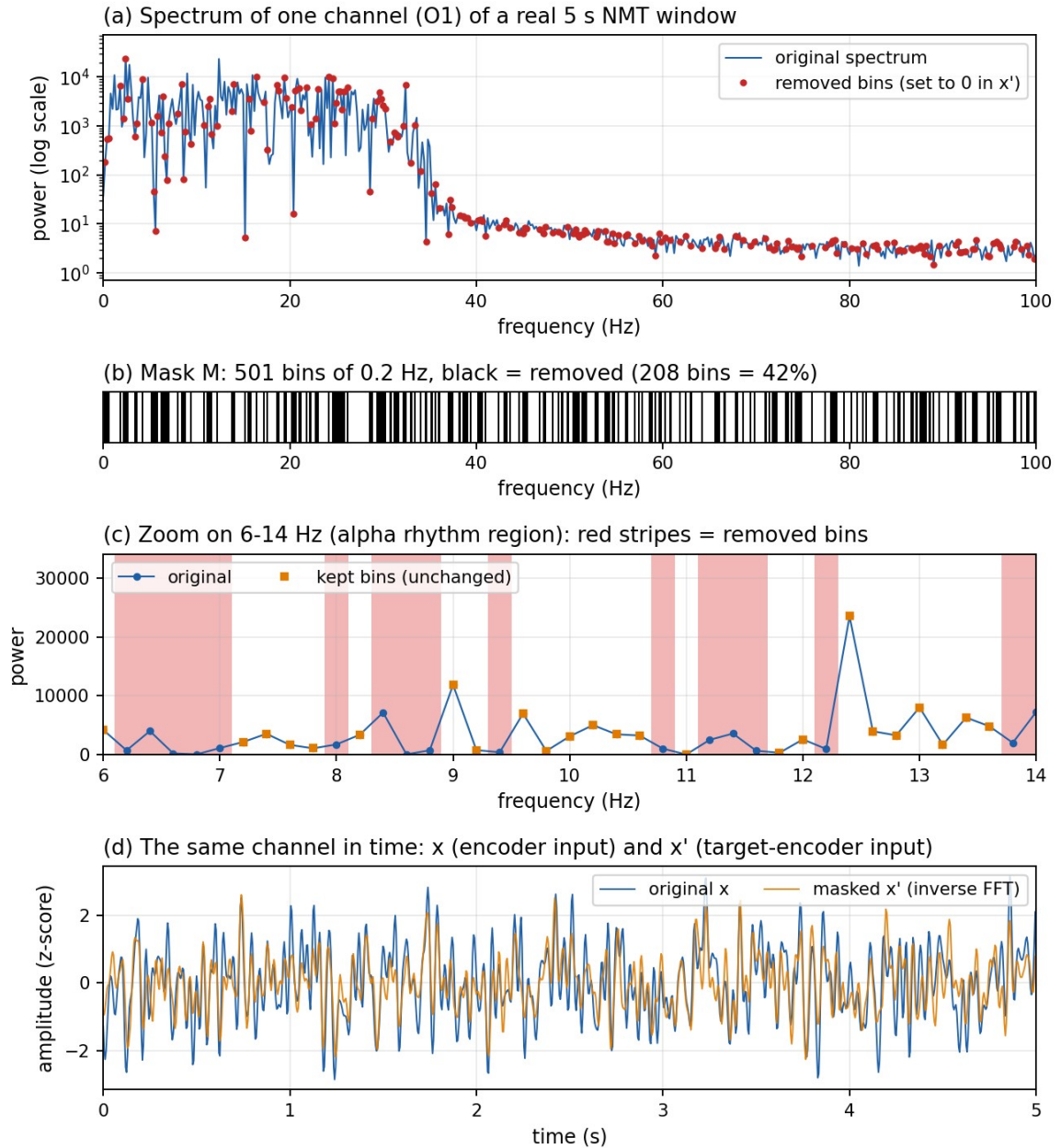
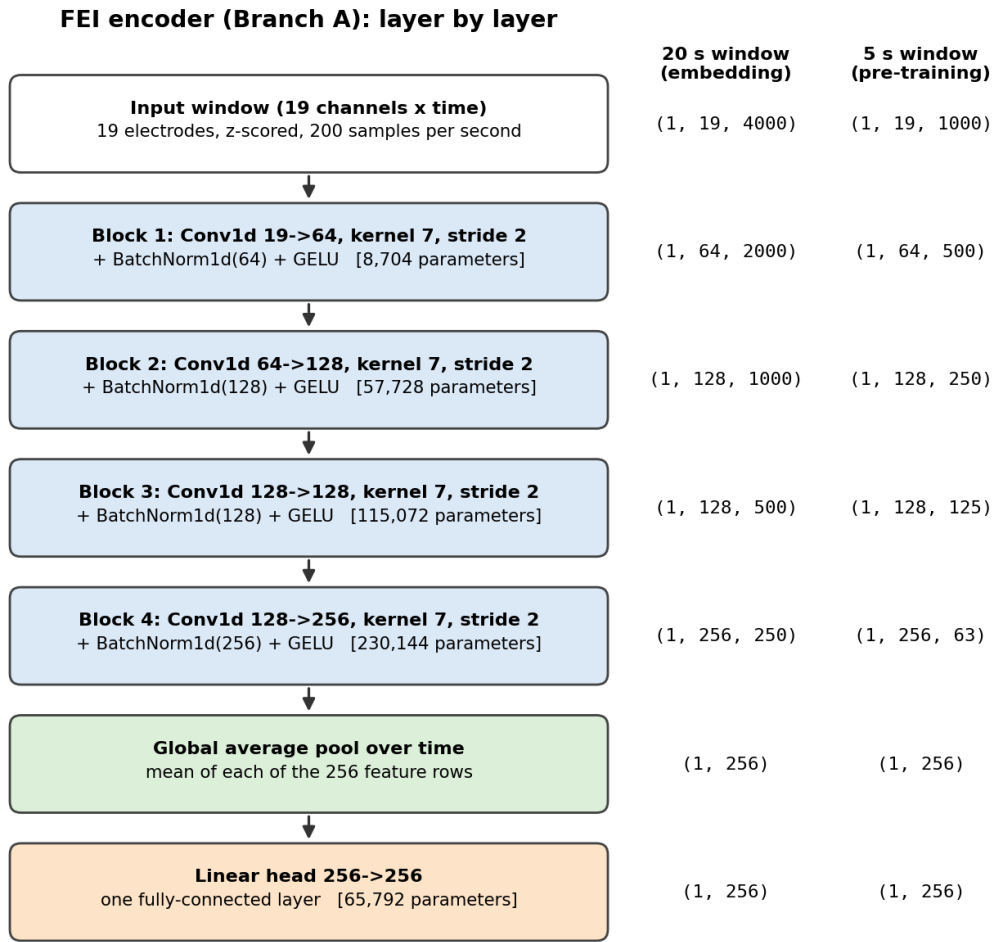


Figure 6: freq_mask applied to one real NMT channel's 5-second window — the original spectrum with the bins a real draw removed marked in red (a), the realized binary mask itself, 208 of 501 bins (42%) removed in this draw (b), a zoom on the alpha band (6-14 Hz) showing exactly which bins survive (c), and the resulting time-domain signal before and after masking and inverse-FFT (d).

Both loss terms in Code Snippet 1 compare *normalized* embeddings, which quietly turns a mean-squared-error loss into a cosine-similarity loss. For unit vectors $\hat{a}$ and $\hat{b}$, $\|\hat{a} - \hat{b}\|^2 = \|\hat{a}\|^2 + \|\hat{b}\|^2 - 2 \cdot (\hat{a} \cdot \hat{b}) = 2 - 2 \cdot \cos(\hat{a}, \hat{b})$, so minimizing `F.mse_loss(nrm(uhat), nrm(up))` is mathematically equivalent to maximizing the cosine similarity between the predicted and target embeddings. This is also why the explicit `F.normalize` call matters: without it, the network could trivially drive the loss towards zero by shrinking its output scale instead of actually learning to predict the masked content's direction in embedding space.

Structurally, the `Encoder` inside FEI is a small 1-D convolutional network rather than a transformer: four blocks, each one `Conv1d` (kernel 7, stride 2) $\rightarrow$ `BatchNorm` $\rightarrow$ `GELU`, with the channel count growing $19 \rightarrow 64 \rightarrow 128 \rightarrow 128 \rightarrow 256$ while stride 2 halves the time length at every block; an average over the remaining time steps then gives 256 numbers, and a final `Linear` layer produces the 256-number embedding used everywhere else in this report.



Output: one embedding of 256 numbers per window, whatever the window length.
 Shapes are (batch, channels or features, time steps), recorded by forward hooks on fei_pretrain.Encoder with checkpoint fei_enc_cv_s0_f0.pt. Each block halves the time axis;
 one step after block 4 sees 91 input samples = 0.46 s. Encoder: 477,440 parameters.

Figure 7: FEI's convolutional encoder — four Conv1d/BatchNorm/GELU blocks taking 19 raw EEG channels down to a 256-number embedding.

Table 1: FEI, layer by layer, with exact parameter counts instantiated from code/fei_pretrain.py at the real Act-0 config (L = 1,000 samples = 5 s at 200 Hz, n_freq=501, d=256, h=128, 19 channels). The target branch is never trained directly — it is an EMA copy of the online branch's weights, so its parameter count matches the online encoder exactly.

Component	Layer	Shape (in → out)	Parameters
Encoder (online + target, shared structure)	Conv1d block 1 + BatchNorm1d	19 → 64, k=7, s=2	8,704
	Conv1d block 2 + BatchNorm1d	64 → 128, k=7, s=2	57,728
	Conv1d block 3 + BatchNorm1d	128 → 128, k=7, s=2	115,072
	Conv1d block 4 + BatchNorm1d	128 → 256, k=7, s=2	230,144
	Global avg-pool + head Linear	256 → 256	65,792
	proj Linear	256 → 128 (h)	32,896
mask_enc	Linear (mask prompt → embedding space)	501 (n_freq) → 128 (h)	64,128

Component	Layer	Shape (in → out)	Parameters
z1 predictor	Linear → GELU → Linear	128 → 128 → 128	33,024
z2 predictor	Linear → GELU → Linear	128 → 128 → 128	33,024
Online branch total	enc + proj (trainable)	—	510,336
Target branch total	enc_t + proj_t (EMA, frozen)	—	510,336
TOTAL	online + target + mask_enc + z1 + z2	—	1,150,848 (1.15M)
TRAINABLE subset	everything except enc_t, proj_t	—	640,512 (0.64M)

FEI is trained with AdamW (learning rate 2×10^{-4} , weight decay 10^{-4}), batch size 128, for 60 epochs, drawing 4 random 5-second windows per recording per epoch (`--wpe` in code/fei_pretrain.py) rather than always the same crop. At probe time, a recording's single 256-number embedding is its online encoder's output mean-pooled over *every* non-overlapping window in that recording (`embed\_recording`), not one arbitrarily chosen window. Because the encoder ends in a global average over time it accepts any window length, and every FEI number in Experiments 5, 6 and 8 embeds 20-second (4,000-sample) windows, the same length GTJ uses, even though FEI is pre-trained on 5-second ones.

GTJ (GRU-based Time-JEPA) targets a different weakness: a hand-crafted band-power feature *averages* spectral power over an entire window, throwing away how that spectrum *evolves* moment to moment. GTJ instead keeps a short-time Fourier transform (STFT) spectrogram (frequency $\times$ time) and trains a GRU-based encoder to predict the embedding of the *future* portion of a recording's spectrogram from the embedding of its *past* portion. Because the future frames are structurally never shown to the context encoder, this pretext task cannot be trivially shortcut the way an earlier masked-input design we tried could (see Novelty, below).

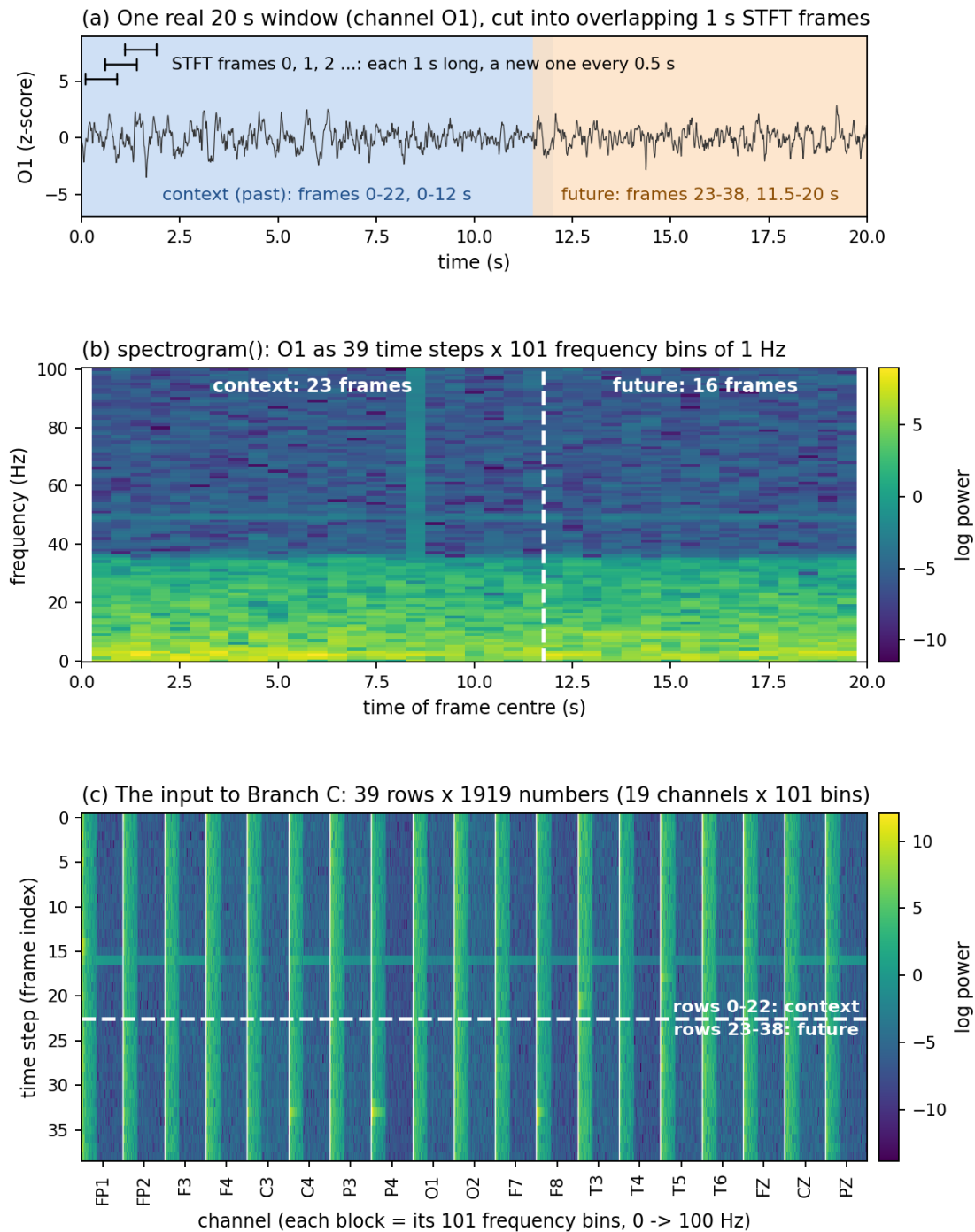


Figure 8: GTJ turns each EEG window into a log-power spectrogram (frequency $\times$ time) and trains a GRU encoder to predict the future span's embedding from the past span's embedding.

```
def spectrogram(x):
    """(B,19,L) -> (B, T, 19*NF) log-power frames: a temporal SEQUENCE of spectra."""
    z = torch.stft(x.reshape(B_*C, L), n_fft=NFFT, hop_length=HOP, win_length=NFFT,
                  window=_WIN.to(x.device), center=False, return_complex=True)
    p = (z.real ** 2 + z.imag ** 2 + 1e-6).log() # log-power
    return p.reshape(B_, C, NF, -1).permute(0, 3, 1, 2).reshape(B_, -1, C * NF)

class DynEncoder(nn.Module):
    """Per-frame spectral embed + GRU over time -> summary of a frame span."""
    def __init__(self, d=256, in_dim=None):
        super().__init__()
        self.bn = nn.BatchNorm1d(in_dim); self.embed = nn.Linear(in_dim, d)
        self.gru = nn.GRU(d, d, batch_first=True)
```

```

    def forward(self, z):
        # z: (B,T,in_dim)
        h = torch.relu(self.embed(self.bn(z.reshape(-1, z.shape[-1])).reshape(z.shape)))
        out, _ = self.gru(h)
        return out[:, -1]
        # last hidden = summary of
the span

class DynJEPA(nn.Module):
    """Predict FUTURE-span embedding from PAST-span embedding. BYOL-style, EMA target."""
    def forward(self, z):
        past, future = z[:, :TC], z[:, TC:]
        u = self.proj(self.enc(past))
        with torch.no_grad():
            v = self.proj_t(self.enc_t(future))
        nrm = lambda a: F.normalize(a, dim=-1)
        return F.mse_loss(nrm(self.pred(u)), nrm(v))

```

Code Snippet 2: GTJ's spectrogram construction and its past-predicts-future JEPA objective (code/fei_branchC.py, simplified). "EMA target" means the target encoder (enc_t, proj_t) is never trained directly: it is an exponential-moving-average copy of the main encoder's weights, updated slowly after every step, which keeps the prediction target stable instead of chasing a network that is changing as fast as the one being trained.

The same idea plays out here as a picture, for one concrete batch: the spectrogram splits into past and future spans, each summarized by the shared DynEncoder, and the past span's predictor output is pushed towards the EMA target's future-span embedding.

GTJ: Spectral-Dynamics JEPA

Predicts the future-span embedding from the past-span context

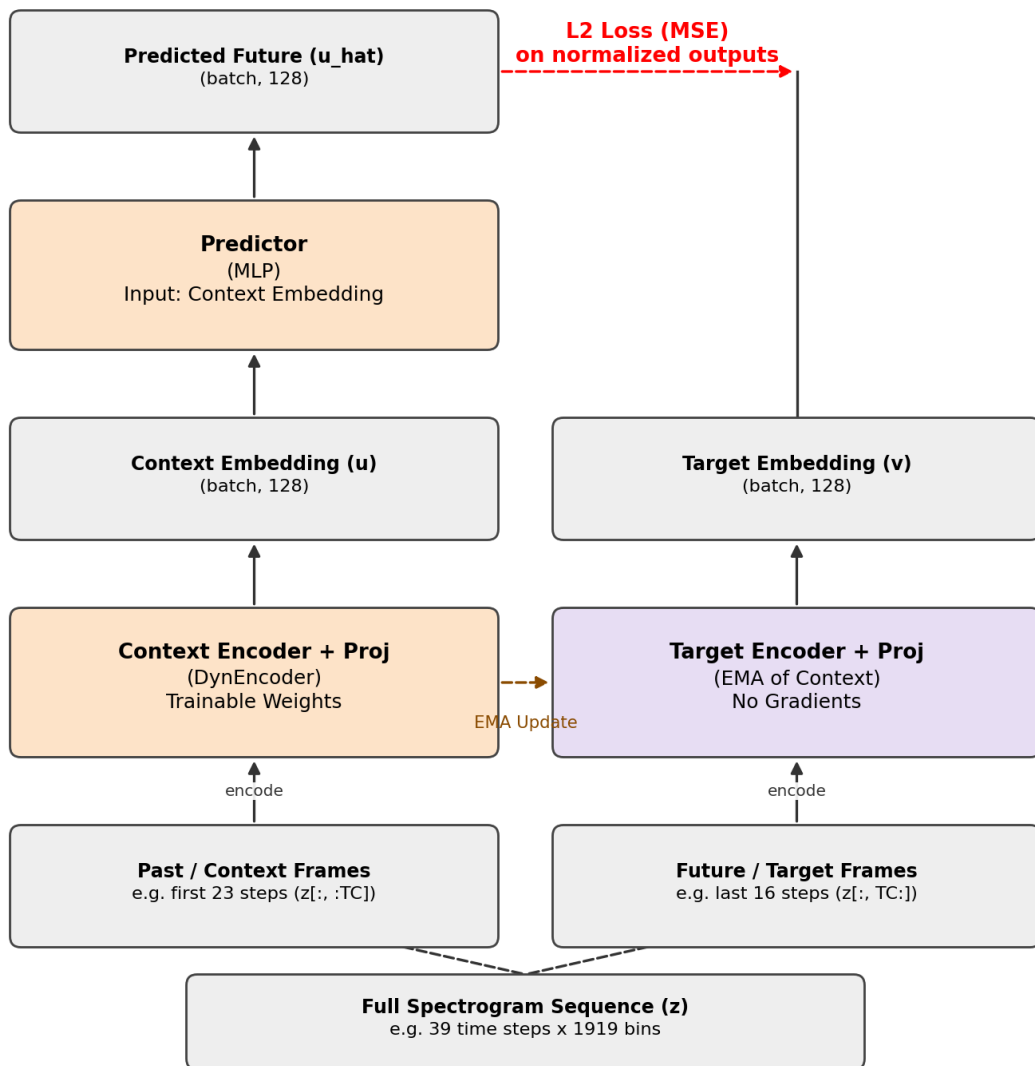
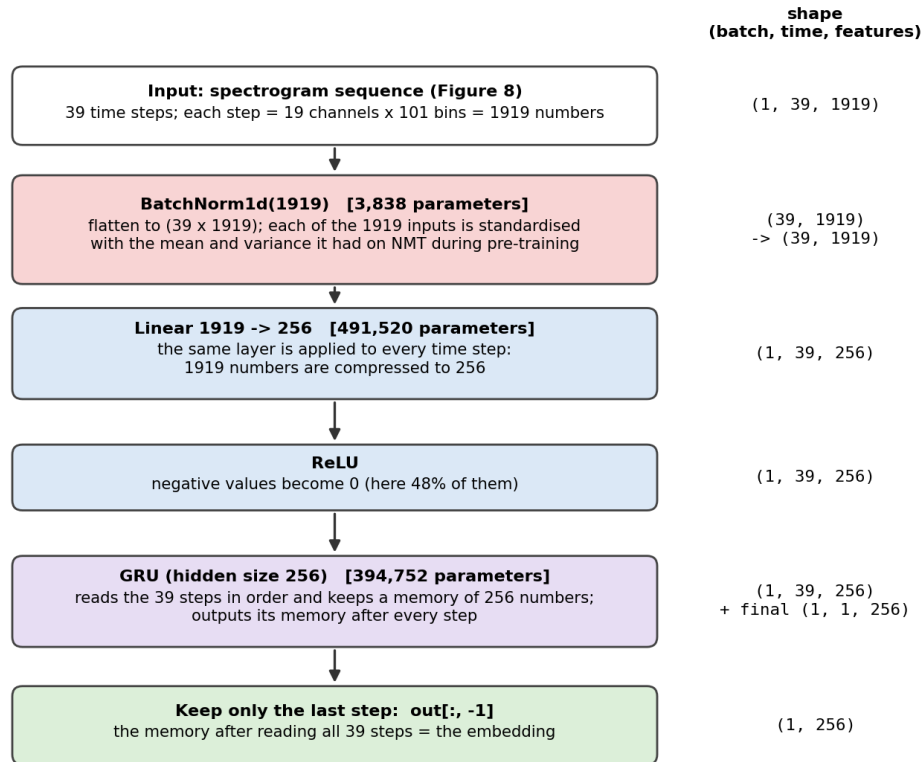


Figure 9: GTJ's past-predicts-future JEPA step — the spectrogram's first 23 frames (past/context) and last 16 frames (future/target) are each summarized by the shared DynEncoder structure (online weights for the past span, the EMA target copy for the future span); a predictor MLP maps the past embedding towards the future embedding, and the normalized squared error between them is backpropagated into the online encoder, with the target encoder trailing it by EMA, exactly as in FEL.

The next figure breaks the DynEncoder itself into its layers, alongside a real example's GRU hidden-unit activations over all 39 frames of one recording's spectrogram.

GTJ encoder (DynEncoder): layer by layer



BatchNorm also stores 3,838 running statistics (mean and variance of each input). They are not trained by gradients; in eval mode they are frozen at their NMT values. Encoder: 890,110 parameters. Shapes recorded by forward hooks on fei_branchC.DynEncoder, checkpoint branchC_h20_enc_cv_s0_f0.pt.

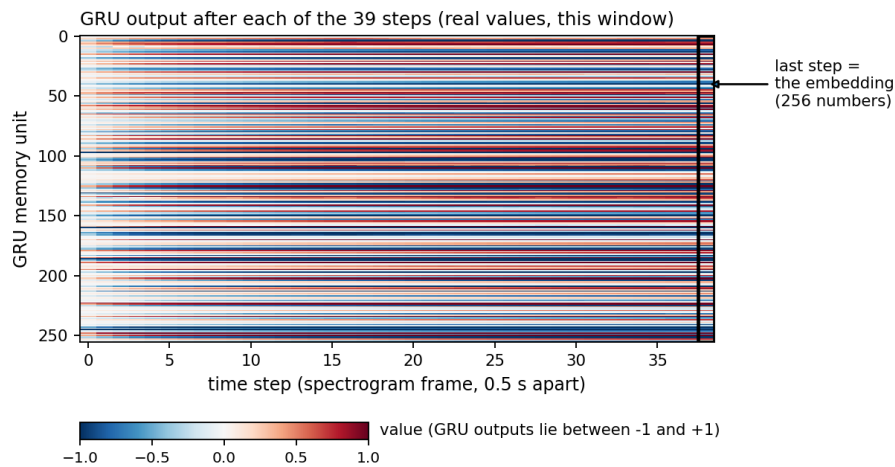


Figure 10: GTJ's DynEncoder, layer by layer — BatchNorm1d over the 1,919-dim input, a Linear embedding down to 256 dimensions with a ReLU, and a 256-unit GRU of which only the last hidden state is kept — alongside a real heatmap of that GRU's hidden-unit activations across all 39 spectrogram frames for one example window, from checkpoint branchC_h20_enc_cv_s0_f0.pt.

Table 2: GTJ, layer by layer, with exact parameter counts instantiated from code/fei_branchC.py at the real Act-0 config (L=4,000 samples = 20 s at 200 Hz, nfft=200, hop=100, giving NT=39 frames, NF=101 frequency bins, a 23-frame past / 16-frame future split). As with FEI, the target branch's parameter count matches the online branch exactly because it is an EMA copy, not an independently trained network.

Component	Layer	Shape (in → out)	Parameters
Input (per window)	Log-power STFT spectrogram	19 ch × 101 freq bins × 39 frames → 1,919-dim/frame	—

Component	Layer	Shape (in → out)	Parameters
DynEncoder (online + target, shared structure)	BatchNorm1d	1,919	3,838
	Linear (embed) + ReLU	1,919 → 256	491,520
	GRU (1 layer, last hidden state)	256 → 256	394,752
	proj Linear	256 → 128 (h)	32,896
pred predictor	Linear → GELU → Linear	128 → 128 → 128	33,024
Online branch total	enc + proj (trainable)	—	923,006
Target branch total	enc_t + proj_t (EMA, frozen)	—	923,006
TOTAL	online + target + pred	—	1,879,036 (1.88M)
TRAINABLE subset	everything except enc_t, proj_t	—	956,030 (0.96M)

GTJ shares FEI's optimizer and schedule exactly (AdamW, learning rate 2×10^{-4} , weight decay 10^{-4} , batch size 128, 60 epochs, 4 random windows per recording per epoch), and a recording's embedding at probe time is likewise its online encoder's output mean-pooled over every non-overlapping 20-second window (`embed_dyn`` in `code/fei_branchC.py`). Table 2's particular window length and recurrent unit were themselves the product of a dose-response check, not an arbitrary choice. Holding everything else fixed and sweeping the window from 5 s to 20 s to 40 s (`branchC_cv_results_{_,h20,h40}.json``), GTJ alone scored 0.820, 0.836 and 0.823 AUROC, and the gap between its learned embedding and a random-init encoder of identical architecture widened monotonically — +0.021 (4/5 folds), +0.038 (5/5), +0.045 (5/5) — the signature of a model actually exploiting temporal dynamics, rather than of an architecture whose untrained transform alone does the work. Swapping the GRU for a long-memory Linear Recurrent Unit (LRU) [12] (`LRUEncoder`` in `code/fei_branchC.py`, a diagonal recurrence  $h_t = \lambda \odot h_{t-1} + \gamma \odot (Bx_t)$  with  $\lambda$  initialised near 1 for long memory) at 40 s left GTJ essentially unchanged ( $0.819 \pm 0.010$  vs. the GRU's  $0.823 \pm 0.009$ , from `branchC_cv_results_h40lru.json``). So the 40 s dip reflects a longer, more stationary window carrying less to predict, not the GRU running out of memory — which is why the cheaper GRU at 20 s is this report's GTJ configuration throughout. The same sweep's fused scores (0.858, 0.870 and 0.863, and 0.861 with the LRU) pair GTJ with the adaptive-masking FEI variant noted under Code Snippet 1 rather than the plain FEI used everywhere else in this report, which is why its 20-second figure (0.870 ± 0.023) differs from Table 11's 0.865. One caveat: the 20-second window was chosen on these same seed-0 folds, with no nested inner split, so GTJ's 20-second numbers carry some selection optimism. They do not depend on it, though: GTJ beats Table 11's band power (0.786) at every window length in the sweep (0.820, 0.836 and 0.823).

Both branches are frozen after pre-training and their 256-dimensional embeddings are concatenated (**FEI+GTJ**, 512-dimensional) before a standard StandardScaler + class-balanced logistic-regression probe is trained on top (the same lightweight, low-capacity classifier used throughout this report for every model), so that comparisons between encoders are never confounded by one model getting a more powerful classifier than another.

3.3 Data and Preprocessing Pipeline

Three datasets carry this report, each playing a distinct role and each prepared with its own matched pipeline so that comparisons stay fair.

Table 3: The three datasets used in Act 0 and their role in the project.

Dataset	Origin / size	Task & role in Act 0	Notes
NMT Scalp EEG [7]	Pakistan (NUST/MH); 2,417 recordings, 415 abnormal	Primary normal-vs- abnormal task; pre-trains and cross-validates FEI, GTJ and FEI+GTJ	Fully open, no licensing restriction

Dataset	Origin / size	Task & role in Act 0	Notes
TUH Abnormal EEG (TUAB) [8]	USA (Temple University Hospital); 2,993 recordings (its 276-recording evaluation split used here)	Cross-hospital transfer test for the released checkpoints and for FEI+GTJ	Our copy is an unofficial Kaggle mirror — see Legal Issues (§5.5.3); used for internal evaluation only
Stroke resting-state EEG [13]	Zenodo record 19599466; 9 stroke vs 6 control (primary cohort), 18 subjects (sensitivity cohort)	Healthy-vs-stroke transfer test, benchmarked against the clinical BSI baseline	Open Zenodo release; primary cohort only reported per team decision

Two preprocessing families run through this report, and Table 4 lists exactly which experiment uses which. The released EEG-VJEPa checkpoints only accept the input they were pre-trained on, so every recording they see is prepared with the paper's own pipeline (§4.1 of arXiv:2507.03633) [1], followed as closely as its text allows (the three points it leaves open are listed after Code Snippet 3): pick and reorder the same 19 standard 10–20 channels, crop to a fixed 5-minute window, band-pass filter to 1–40 Hz, resample to 100 Hz, z-score each channel, and cut the signal into 5-second frames with 50% overlap (Code Snippet 3). The TUAB band-power baseline of Experiments 4 and 6 reads the same 100 Hz frames. FEI and GTJ instead work at 200 Hz, NMT's native rate: on NMT they read recordings that are only per-channel z-scored (no band-pass, no resampling) and stored as 2.5-second frames; on TUAB and the stroke cohort each recording is resampled to 200 Hz and z-scored, with a 0.5–40 Hz band-pass on TUAB, and a common average reference plus the spectral-matching step described below on stroke. We deliberately re-derived the 100 Hz pipeline from the paper's description rather than assuming our own earlier 200 Hz preprocessing would be compatible with the released checkpoints. An early, since-resolved concern in this project was whether a sample-rate mismatch could explain any gap to the paper; it could not, but getting the released checkpoints' expected input exactly right was an essential and non-obvious first step.

The z-score step is a per-channel, per-recording standardization: for each channel's raw samples x , $z = (x - \mu) / \sigma$, where μ and σ are that channel's own mean and standard deviation over the cropped recording (substituting $\sigma = 1$ on the rare flat channel where $\sigma = 0$, to avoid dividing by zero). Standardizing per recording rather than with a fixed, dataset-wide μ/σ matters here because electrode impedance and amplifier gain vary enough between recordings, and between the NMT and TUAB hospitals in particular, that a single global scale would leave some recordings under- or over-saturated relative to what the encoder saw during pre-training.

```
# Paper preprocessing (arXiv:2507.03633 §4.1), as closely as its text allows:
CHANNELS = ['FP1', 'FP2', 'F3', 'F4', 'C3', 'C4', 'P3', 'P4', 'O1', 'O2',
            'F7', 'F8', 'T3', 'T4', 'T5', 'T6', 'FZ', 'CZ', 'PZ'] # this order
TARGET_FS, WINDOW_SIZE = 100, 500 # 100 Hz, 5 s frames
CROP_SECONDS = 300 # crop to a fixed 5-minute length
BANDPASS = (1.0, 40.0) # bandpass 1-40 Hz (encoder was pretrained on this band)
WINDOW_STRIDE = 250 # 50% overlap -> (30000-500)//250 + 1 = 119 frames (paper: 118)

def preprocess_edf(edf_path, out_path, crop_start=0.0):
    raw = mne.io.read_raw_edf(edf_path, preload=False, verbose=False)
    raw.pick_channels(CHANNELS); raw.reorder_channels(CHANNELS)
    t0 = min(crop_start, max(0.0, raw.times[-1] - CROP_SECONDS))
    raw.crop(tmin=t0, tmax=min(t0 + CROP_SECONDS, raw.times[-1]))
    raw.load_data(verbose=False)
    raw.filter(BANDPASS[0], BANDPASS[1], verbose=False)
    if round(raw.info['sfreq']) != TARGET_FS:
        raw.resample(TARGET_FS)
    data = raw.get_data()
    mean, std = data.mean(1, keepdims=True), data.std(1, keepdims=True)
    data = (data - mean) / np.where(std == 0, 1.0, std) # per-channel z-score
    num_frames = (data.shape[1] - WINDOW_SIZE) // WINDOW_STRIDE + 1
    frames = np.stack([data[:, i*WINDOW_STRIDE : i*WINDOW_STRIDE+WINDOW_SIZE]
                       for i in range(num_frames)]).astype(np.float32)
    np.save(out_path, frames) # atomic (tmp + os.replace)
in full code
```

Code Snippet 3: The paper's own §4.1 preprocessing [1], which prepares every recording the released checkpoints see (code/preprocess_nmt.py, simplified; the stroke cohort gets the same filtering, resampling, z-scoring and framing in code/stroke_phaseB.py, after a common average reference).

Three points are left open by §4.1, and Code Snippet 3 settles each one. It filters at the native sampling rate and then resamples, where the paper lists downsampling before filtering; it z-scores each channel over the whole cropped recording, where the paper's "last step" could also be read as per window (both readings give the same collapsed output from the released ViT-M, Experiment 3); and it keeps every full 500-sample frame at a 250-sample stride, which gives $(30,000 - 500) / 250 + 1 = 119$ frames per 5-minute crop rather than the 118 the paper states. All 2,993 preprocessed TUAB files have shape $119 \times 19 \times 500$. Filtering and resampling use MNE 1.8.0's defaults (a zero-phase FIR band-pass, then FFT-based resampling). A recording shorter than 5 minutes is kept at its full length, and one missing any of the 19 channels fails channel selection and is skipped (the script logs it and moves on). The released encoder reads clips of 32 frames spread evenly over 96 consecutive frames, so a 119-frame recording offers 23 clip positions. It also cuts each 19×500 frame into patches of 4 channels $\times$ 30 samples, so only the first 16 channel rows and the first 480 samples are ever embedded: in the paper's channel order the three midline channels, FZ, CZ and PZ, come last and never reach the released encoder.

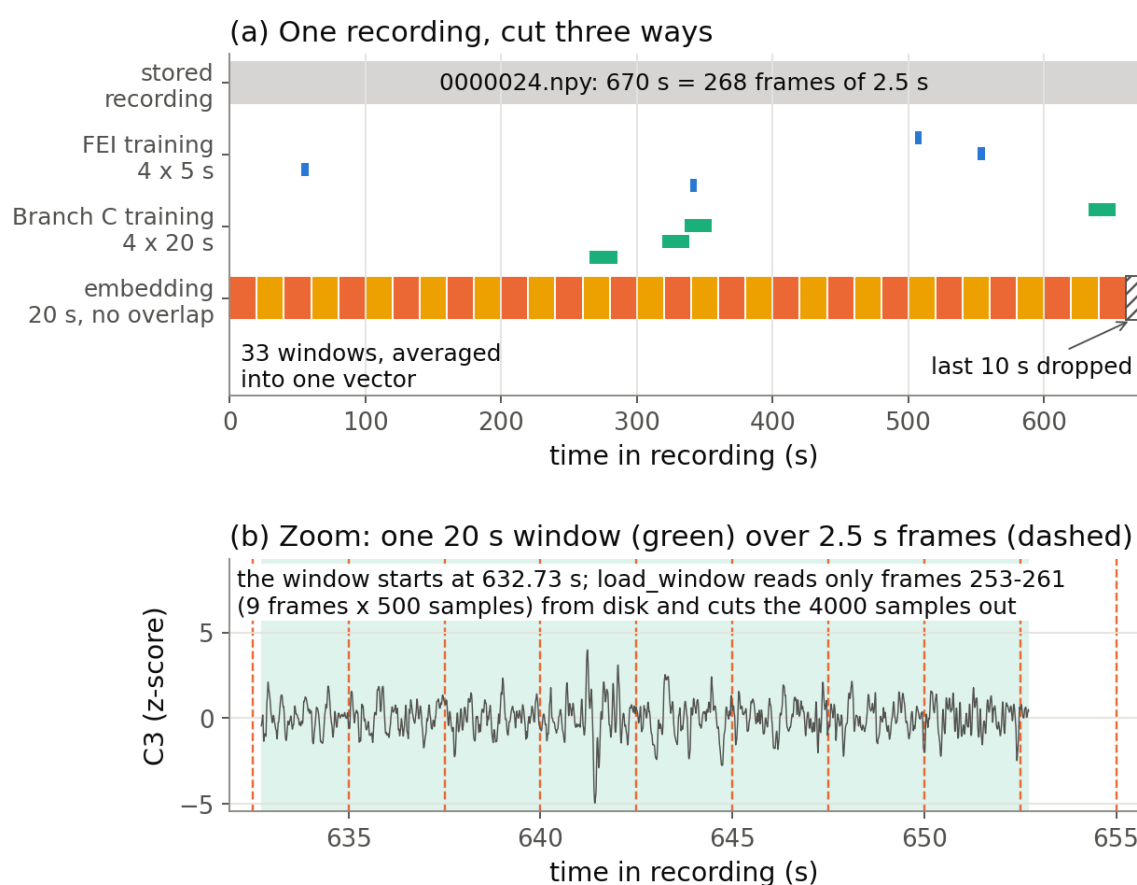


Figure 11: Cropping a continuous EEG recording into fixed-length, 50%-overlapping windows for the encoder input.

The stroke cohort also needed two preprocessing steps the other two datasets do not. First, a purely data-quality step: any channel the dataset's own authors had flagged as bad is rebuilt from its neighbours by spherical-spline interpolation before anything else is done to the recording. Second, a non-obvious step described in full in §5.3.3 (Experiment 8): because its recording hardware and montage differ from NMT's, its raw spectrum does not match what our NMT-trained encoders expect to see, and GTJ in particular is sensitive to this mismatch. We built a label-free, per-channel spectral-matching filter that maps the stroke cohort's mean spectrum onto NMT's before any encoder sees it, fixing this without ever looking at the stroke/control labels. The filter is fitted once on all 15 subjects pooled, so it is transductive (each held-out subject helps shape the filter applied to it), and it was added after the pre-registered analysis had been run; Appendix §11.2 gives the record and a fold-local check. Here and throughout, "pre-registered" means a locally timestamped pre-analysis

protocol (Appendix §11.2), not an entry in a public registry.

Table 4: Which preprocessing feeds which experiment. The released checkpoints and the TUAB band-power baseline use the paper's 100 Hz pipeline; FEI, GTJ and the NMT band-power arm use 200 Hz input. All encoder inputs are per-channel z-scored.

Model / experiment	Dataset	Sampling rate, filtering	Window	Samples / window	Overlap	Script
Released ViT-M/ViT-B (Exps 1–4); band power (Exps 4, 6)	TUAB	100 Hz; 1–40 Hz; first 5 min	5 s	500	50% (2.5 s stride)	preprocess_tuab.py (calls preprocess_nmt.py)
Released ViT-M (Exp 3; Exp 8 comparison)	Stroke	100 Hz; 1–40 Hz; common average reference	5 s	500	50% (2.5 s stride)	stroke_phaseB.py
FEI / GTJ pre-training	NMT (Exp 6 GTJ and fused model: + unlabelled TUAB-train)	200 Hz	5 s (FEI) / 20 s (GTJ)	1,000 / 4,000	random crops, 4 per recording per epoch	fei_pretrain.py / fei_branchC.py (Exp 6 joint: feic_joint.py); NMT input: preprocess_nmt_200hz.py
FEI, GTJ, FEI+GTJ (Exp 5)	NMT	200 Hz; z-scored only, no band-pass	20 s	4,000	none	feic_nmt_anchor.py
Band power (Exp 5)	NMT	200 Hz; z-scored only; power in 0.5–45 Hz bands	whole recording (Welch)	500 per Welch segment	50% between Welch segments	baseline_bandpower.py, act0_nmt_bandpower.py
FEI, GTJ, FEI+GTJ (Exp 6)	TUAB	200 Hz; 0.5–40 Hz; first 20 min	20 s	4,000	none	tuab_feic.py
FEI, GTJ, FEI+GTJ (Exp 8)	Stroke	200 Hz; common average reference, no band-pass; spectrum matched to NMT	20 s	4,000	none	stroke_phaseB_clean.py
BSI and spectral features (Exp 7)	Stroke	1,000 Hz native; A2 reference; 50 Hz notch; 0.5–100 Hz	2 s epochs	2,000	none (epochs over ± 100 μ V dropped)	stroke_phaseB.py

3.4 Core Source Code and Reproducibility

The full codebase (all pre-training, evaluation, statistics, and plotting scripts used for every number and figure in this report) is organized as one script per experiment, in the repository github.com/mashfiquurrahman2020-coder/EEG-VJEPa-act-0. Table 5 maps each report section to the script that produced it. The code snippet below shows the check we relied on most: compare a pre-trained model's score with a random-initialisation control run through the identical code path.

Table 5: Map from report section to the script that produced its numbers.

Script	Purpose
preprocess_nmt.py	The paper's §4.1 pipeline (crop, 1–40 Hz filter, resample to 100 Hz, z-score, window) for released-checkpoint input
preprocess_tuab.py	Runs TUAB through preprocess_nmt.py's §4.1 pipeline (Experiments 1–4, and the band-power baseline of Experiments 4 and 6)
preprocess_nmt_200hz.py	The 200 Hz NMT input FEI and GTJ read (recipe reconstructed after the original script was lost; reproduces all 2,417 stored files byte for byte)
fei_pretrain.py	FEI model, frequency masking, pre-training loop, 5-fold CV driver

Script	Purpose
fei_branchC.py	GTJ model, spectrogram construction, past-future JEPA, fusion CV driver
feic_joint.py	FEI and GTJ re-pre-trained on NMT plus unlabelled TUAB-train (Experiment 6's GTJ and fused rows)
feic_nmt_anchor.py	FEI, GTJ and FEI+GTJ on NMT, seed-0 5-fold CV (Experiment 5)
act0_nmt_rand_controls.py	Random-init twins of FEI, GTJ and FEI+GTJ on Experiment 5's folds (5 seeds)
act0_nmt_pretrain_replicate.py	Two further pre-training runs of FEI and GTJ per fold, with new seeds (Experiment 5)
act0_nmt_stats.py	Nadeau–Bengio corrected t-tests with Holm adjustment for Experiment 5 (Table 20)
baseline_bandpower.py	NMT relative band power at 200 Hz (Experiment 5)
act0_nmt_bandpower.py	NMT band power on Experiment 5's exact folds, with paired tests
tuab_bandpower.py	Hand-crafted relative band-power baseline on TUAB's 100 Hz frames (Experiments 4, 6)
tuab_released_eval.py / tuab_cached_eval.py	Released ViT-M/ViT-B frozen evaluation on TUAB (Experiment 1)
tuab_finetune_fast.py	ViT-M fine-tuned from the released weights and trained from scratch, with the authors' recipe (Experiment 2)
tuab_vitm_linear.py	Linear probe on frozen released and random-init ViT-M (Experiment 4; the ViT-M row of Experiment 6)
tuab_collapse_check.py	Token-std / cross-recording-cosine collapse diagnostics (Experiment 3)
tuab_feic.py	FEI+GTJ transfer evaluation on TUAB, with random-init controls (Experiment 6)
tuab_fei_reps.py	FEI's NMT-only TUAB gain over five pre-training runs, two of them pre-trained here, with the multi-run bootstrap (Experiment 6)
tuab_feic_joint_reps.py	Two seeded re-runs of Experiment 6's adapted (NMT+TUAB) FEI and GTJ pre-training, scored as in tuab_feic.py
stroke_phaseB.py / stroke_phaseB_cv.py	BSI baseline + LOSO / 5-fold×100 / leave-pair-out CV machinery
stroke_phaseB_clean.py	Spectrum-matching fix and its input/embedding health gates (Experiment 8)
stroke_phaseB_foldlocal.py	Stroke spectral-matching filter refitted inside each LOSO fold (Table 19)
stroke_phaseB_reps.py	Experiment 8 scored with Experiment 5's two seeded pre-training re-runs
act0_repro_manifest.py	Fold, subset and cohort lists; checkpoint and result hashes; environment (Appendix §11)
act0_raw_hashes.py	SHA-256 of every raw recording (Table 21)
act0_metrics.py	Shared metric definitions and bootstrap confidence intervals
act0_full_metrics.py	BACC, AUC-PR, F1, sensitivity and specificity columns, recomputed from each experiment's saved predictions
act0_make_results.py	Experiments 1, 2 and 4's Mann–Whitney, paired t- and Wilcoxon tests, from the saved results
failure_axes.py	Per-recording failure-axis analysis from saved out-of-fold predictions

One formula named in Table 5 but never actually shown: the classical band-power baseline behind every “band power” row in Experiments 4 and 6. For each channel, Welch’s method estimates a power spectral density from the already-filtered, already-windowed signal; summing that density inside five fixed bands and dividing by their total gives each band’s *relative* power — a channel’s spectral shape with its absolute amplitude divided out, which keeps the comparison fair on top of input that is already per-recording z-scored (§3.3). Experiment 5’s NMT band-power arm applies the same construction to NMT’s own 200 Hz input (code/baseline_bandpower.py): Welch over the whole recording rather than per frame, with bands 0.5–4, 4–8, 8–13, 13–30 and 30–45 Hz, normalised by total 0.5–45 Hz power.

```
BANDS = [(1, 4), (4, 8), (8, 13), (13, 30), (30, 40)] # delta/theta/alpha/beta/gamma; data
already 1-40 Hz band-passed
FS = 100 # Hz, matching the shared
preprocessing pipeline (Code Snippet 3)

def rbp(path):
    x = np.load(path) # (119, 19, 500): 5 s frames, 50%
    overlap
    f, p = welch(x, fs=FS, nperseg=250, axis=-1) # Welch PSD per frame
    p = p.mean(0) # average over the 119 frames ->
    (19, freqs)
    bp = np.stack([p[:, (f >= lo) & (f < hi)].sum(-1) for lo, hi in BANDS], -1)
    return (bp / bp.sum(-1, keepdims=True)).ravel() # (95,) relative band power = 19
    channels x 5 bands
```

Code Snippet 4: The classical band-power baseline’s exact feature, used for every TUAB band-power number, in Experiments 4 and 6 (code/tuab_bandpower.py, simplified).

```
def fit_probe(Xtr, ytr, Xte, yte):
    """The ONE probe used for every encoder in this report: StandardScaler + balanced
    LogReg, C=1."""
    sc = StandardScaler().fit(Xtr)
    clf = LogisticRegression(max_iter=2000, class_weight="balanced",
    C=1.0).fit(sc.transform(Xtr), ytr)
    s = clf.decision_function(sc.transform(Xte))
    return roc_auc_score(yte, s), balanced_accuracy_score(yte, (s > 0).astype(int))

# The habit that runs through every experiment in this report:
# score_pretrained = fit_probe(embed(pretrained_encoder, ...))
# score_random = fit_probe(embed(RANDOM-INIT, same architecture, same pipeline))
# -> only trust "pre-training helped" if score_pretrained beats score_random, with a
stated p-value.
```

Code Snippet 5: The single shared evaluation probe, and the random-init-control discipline applied to every pre-trained encoder in this report (the one exception is disclosed in Experiment 1).

4 Implementation

4.1 Description

Nothing was combined before it was tested on its own. In the weeks before the mid-term break (around week 7), FEI was built and unit-tested in isolation: its self-test checks that masking a signal’s 10 Hz component actually nulls a 10 Hz test tone, and that the JEPA loss drops over a few optimisation steps on a synthetic batch, before any real EEG reaches it. GTJ got the same treatment in week 8. Its self-test checks spectrogram shapes, checks the loss drops, and checks that the past and future halves of a spectrogram aren’t numerically identical; otherwise “predict the future” is trivially solvable and means nothing.

One data-loading problem showed up before either encoder was even running: an early version of the NMT loader pulled every recording fully into memory and hit a 24 GB out-of-memory error. Switching to a memory-mapped loader fixed that (down to about 2 GB), and, as a side effect, training ran roughly ten times faster, since the GPU was no longer waiting on disk reads. We should have done that from day one.

Only once FEI and GTJ had each passed their own self-tests and their own single-branch NMT cross-validation (Experiment 5) were they fused into FEI+GTJ. The same build-alone-then-integrate pattern repeated for every later addition. The band-power and BSI baselines were checked against known clinical effects before

being used as a comparison bar (the pipeline-soundness gate in Experiment 7).

Every long pre-training run checkpoints its state to disk after each cross-validation fold. The shared GPU has, more than once, needed to be freed for another job, or paused outright by the thermal guard; a resumed run just picks up from the last completed fold instead of starting over.

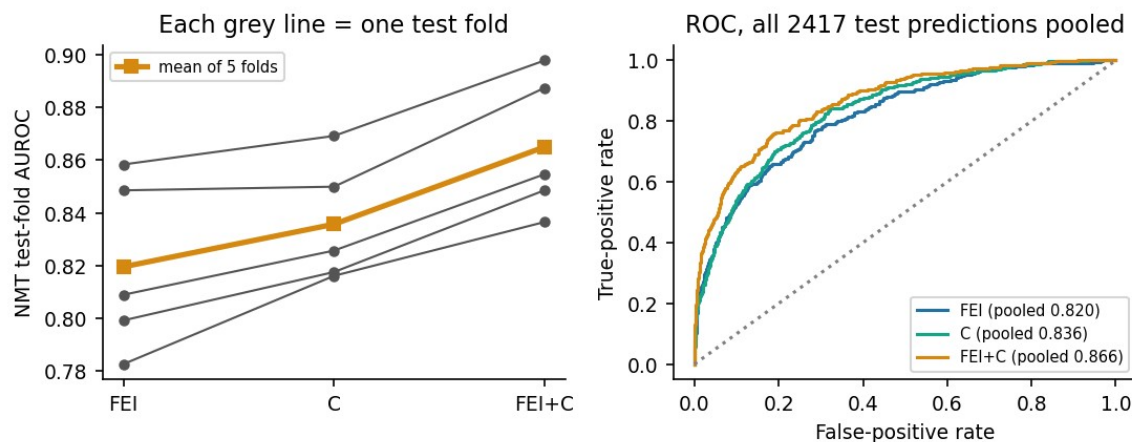


Figure 12: Assembling FEI and GTJ into FEI+GTJ on NMT — each grey line is one cross-validation fold; fusing the two branches raises every single fold's AUROC over either branch alone.

5 Design Analysis and Evaluation

5.1 Novelty

Neither FEI nor GTJ is architecturally new by itself. Removing random frequency components of a time series is already a pre-training augmentation in time-frequency-consistency contrastive learning [14]; FEI's twist is to predict, in embedding space, how a momentum target encoder sees the masked copy. Predicting a signal's future from its past is what contrastive and autoregressive predictive coding already do for speech [15, 16], and contrastive predictive coding has already been applied to clinical EEG, including pathology detection [17]. Two recent EEG papers come closer still: one pre-trains by autoregressively predicting future spectro-temporal content [18], and a 2026 preprint that appeared during this project predicts future latent EEG states with an EMA target encoder [19].

We found that a specific, publicly released EEG foundation-model checkpoint is representation-collapsed. We could not find this diagnosis published at the time. Token standard deviation and cross-recording cosine similarity make it simple enough to re-run on that checkpoint in minutes.

FEI+GTJ is our working two-branch replacement. An earlier GTJ design masked out spectrogram regions and asked the encoder to reconstruct them, and Ahmed found it could be solved almost entirely from each window's average spectral content: the network wasn't learning much more than the band-power baseline already in this report. Moving to a past-predicts-future design, where the future half is never shown to the context encoder at all, closed that shortcut. The resulting pair is validated end-to-end, with paired statistics, across three datasets and two clinically distinct tasks.

GTJ later collapsed on a different dataset because its BatchNorm layers carried NMT's input statistics across domains. We diagnosed that failure and corrected it without labels. The fix may generalise to other EEG self-supervised encoders that normalise their input the same way, but we have tested it only on the cohort it was designed on.

5.2 Design Considerations

5.2.1 Considerations to public health and safety

A model that flags EEG recordings as “abnormal” touches patient safety directly. A false negative could delay a diagnosis. A false positive could trigger unnecessary follow-up tests, or just worry someone unnecessarily. Our failure-axis analysis (§5.3.4) found error rates are not uniform across age groups: FEI over-flags healthy young children as abnormal at more than six times the rate it does healthy adults. A system with that gap has

no business sitting near a real clinical decision for a paediatric patient, whatever its average AUROC says.

5.2.2 Considerations to environment

Every model here is intentionally small (1.15M parameters for FEI and 1.88M for GTJ, counting both the online and the EMA target copy; Tables 1–2) and trained for tens, not hundreds, of epochs on a single consumer GPU, rather than the institutional clusters that CBraMod and LaBraM pre-train on. Partly this was just what hardware we had. But it's also a real result: a model sized to match its dataset, instead of scaled up regardless of how much data is available, already beats a classical baseline and, with TUAB's unlabelled recordings added to pre-training, scores close to a released large-scale checkpoint under the same linear probe on TUAB (0.862 vs 0.838, not significantly different). Every run is also gated by a thermal guard (§5.4) that pauses or aborts a job if CPU temperature crosses a safe threshold, which caps energy draw as a side effect.

5.2.3 Considerations to cultural and societal needs

NMT, which underlies most of this project, was collected in Pakistan specifically to give under-represented, lower-resource clinical populations a presence in EEG machine-learning research. Most large datasets, TUAB included, come from well-resourced North American and European hospitals. Showing that a small encoder built on this dataset transfers its pre-training benefit to a different hospital's data (§5.3.3, Experiment 6) is one small piece of evidence that self-supervised EEG models don't have to depend on high-resource-setting data alone. Stroke, the third task here, is one of the leading causes of long-term disability worldwide; low-cost, automatable screening matters most exactly where specialist neurological review is hardest to get.

5.3 Investigations

5.3.1 Design of Experiment

Every experiment here follows the same rules. Any claim that pre-training “helped” needs a paired comparison against the same architecture with random, untrained weights, run through identical evaluation code. Every formal inferential comparison between two models comes with a confidence interval or a p-value: bootstrap resampling for independent recordings, paired tests for same-subset comparisons (for cross-validation folds, whose training sets overlap, the Nadeau–Bengio corrected resampled t-test [20], with Holm's adjustment [21] across a family of comparisons), label-permutation tests for small cohorts, whichever fits the data at hand. And experiments are ordered by increasing domain shift: validate on the data an encoder was trained on first (NMT), then transfer it, frozen, to a different hospital's data (TUAB), and only then apply it to a structurally different clinical task it was never designed for (stroke vs. healthy). The TUAB step is run with all three encoders pre-trained on NMT only, and again for GTJ and the fused model after adding TUAB's unlabelled training recordings to pre-training (Experiment 6). Working at the first, easiest step is a much weaker claim than surviving all three.

Two recurring checks in this design are, in effect, this report's ablation study: the random-init control run alongside almost every pre-trained encoder (removing “pre-training” as the only variable, with architecture and evaluation code held fixed) and Experiment 4's comparison across feature/probe choices (removing or swapping the encoder itself, with the classifier held fixed). Rather than a single dedicated ablation table, this project's ablations are distributed across the experiments below at the point where each one is most informative.

5.3.2 Data Collection

Dataset acquisition and preparation are described in §3.3 above; Table 3 there has size, role, and any licensing caveats for all three datasets.

5.3.3 Results and Analysis

We use AUROC everywhere. It doesn't care about the decision threshold, which matters because we never settled on one. Balanced accuracy and AUC-PR show up alongside it where useful. Unless stated otherwise, “±” is the standard deviation across cross-validation folds or training subsets, and bracketed ranges are 95% bootstrap confidence intervals.

Formally, writing TP/FP/TN/FN for true/false positives/negatives at a given threshold, sensitivity (true-

positive rate) is $\text{TPR} = \text{TP} / (\text{TP} + \text{FN})$ and specificity's complement (false-positive rate) is $\text{FPR} = \text{FP} / (\text{FP} + \text{TN})$; **AUROC** is the area under the curve traced by (FPR, TPR) as the decision threshold sweeps over every possible value, equivalently the probability that a randomly chosen abnormal recording is scored higher than a randomly chosen normal one. **Balanced accuracy** fixes a single threshold (predicted probability $p > 0.5$ throughout this report, equivalently $\text{decision_function} > 0$ for the logistic-regression probe of Code Snippet 5, which is scikit-learn 1.5.2's LogisticRegression with $C = 1$, balanced class weights and otherwise default settings: L2 penalty, lbfgs solver, tolerance 10^{-4}) and averages the two classes' recall so neither is drowned out by the other's size: $\text{BAcc} = \frac{1}{2} \cdot (\text{TPR} + \text{TNR})$, where $\text{TNR} = \text{TN} / (\text{TN} + \text{FP})$. **AUC-PR** is the same sweep-every-threshold idea as AUROC but traces precision = $\text{TP} / (\text{TP} + \text{FP})$ against recall = TPR instead of FPR against TPR, which makes it more sensitive to performance on the minority (here, abnormal or stroke) class than AUROC is.

Experiment 1 — Frozen reproduction of the paper's TUAB result.

The most basic question came first: load the authors' own released checkpoints (SHA-256-verified against the Hugging Face file records; hashes in Table 18) into the authors' own evaluation heads, run them on TUAB v3.0.1's 276-recording evaluation split, the one the paper reports on (files from our mirror copy, §5.5.3), and see whether we land anywhere near their reported 0.877 (ViT-M) / 0.879 (ViT-B) AUROC. To rule out our own code before blaming theirs, Maruf diffed our pipeline against the authors' public repository (github.com/amir-hojjati/eeg-vjepa) at commit `c739fad5ad57ec4439a094343901a245edf611ec`: 43 of 48 compared files were byte-identical. The other 5 (`app/main.py`, `app/vjepa/train.py`, `eval.py`, `schedulers.py` and `attentive_pooler.py`) carry clearly-labelled additions of ours — a thermal guard, a fixed evaluation seed, an optional alternate head, an optional learning-rate scale, balanced accuracy among the metrics — that don't change the computation at their defaults.

The same evaluation heads recur throughout the experiments below — a Linear Probe, the paper's own §4.2 Attention Classifier, and its Fig-3 Attentive Probe [1] — each sitting on top of a frozen (or, in two cases, fine-tuned) encoder and turning its output into a normal/abnormal decision.

Downstream Linear Probe Architecture

Used for frozen evaluation on TUAB

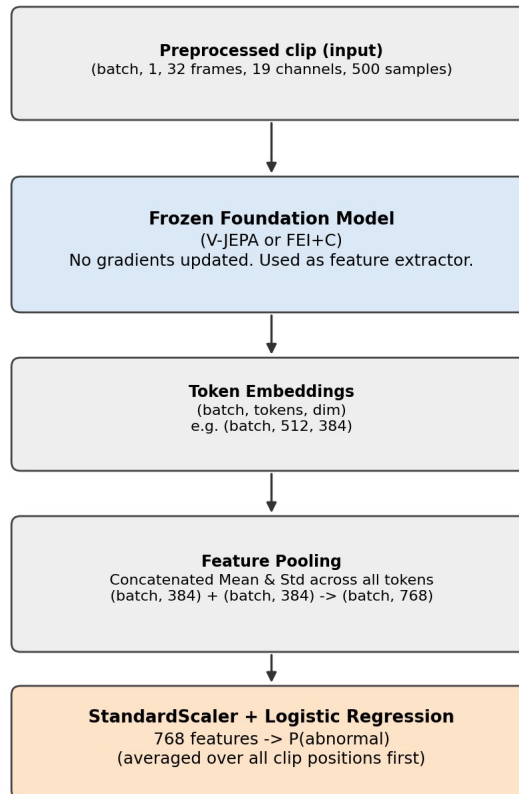


Figure 13: The Linear Probe — mean and standard deviation pooled over all tokens (or, for FEI/GTJ, the raw embedding), then a StandardScaler + logistic regression.

The Linear Probe is the simplest of the three: for ViT-M, it takes the mean and standard deviation of the 512 output tokens ($384 + 384 = 768$ numbers), averaged over every clip position in a recording; for FEI/GTJ, it is simply their 256(+256)-number embedding directly. Either way, a single `StandardScaler` + class-balanced `LogisticRegression` sits on top (the same lightweight head used for every encoder in this report), so that no comparison between models is confounded by one getting a more powerful classifier than another.

Downstream Attention Classifier (Fine-Tuning)

Used for end-to-end fine-tuning on TUAB

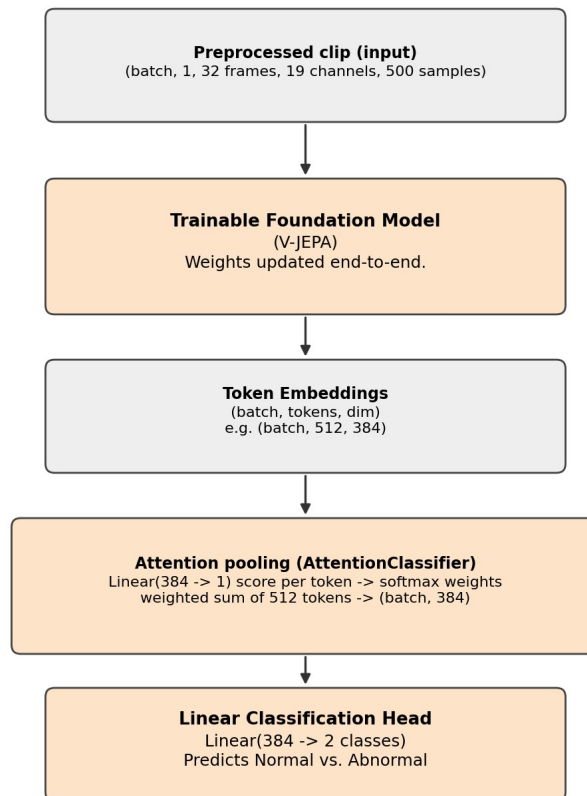


Figure 14: The paper's §4.2 Attention Classifier — a learned score per token, softmax-normalised into weights, a weighted average, then a linear classifier [1].

The §4.2 Attention Classifier (`AttentionClassifier` in the authors' own code) is the head the paper itself evaluates with, and the only one of the three ever used for fine-tuning. It has just two learned layers: a Linear(384 → 1) scores every one of the 512 tokens, softmax turns those 512 scores into weights that sum to 1, the tokens are averaged with these weights into a single 384-number vector, so the head effectively learns *which tokens to listen to*, and a final Linear(384 → 2) reads off the normal/abnormal decision. We use it both frozen (on the released checkpoint) and as the head trained end-to-end during fine-tuning and from-scratch training (Experiment 2) [1].

Attentive Probe (AttentiveClassifier, Fig-3 head)

Frozen evaluation on TUAB only; ViT-M sizes shown

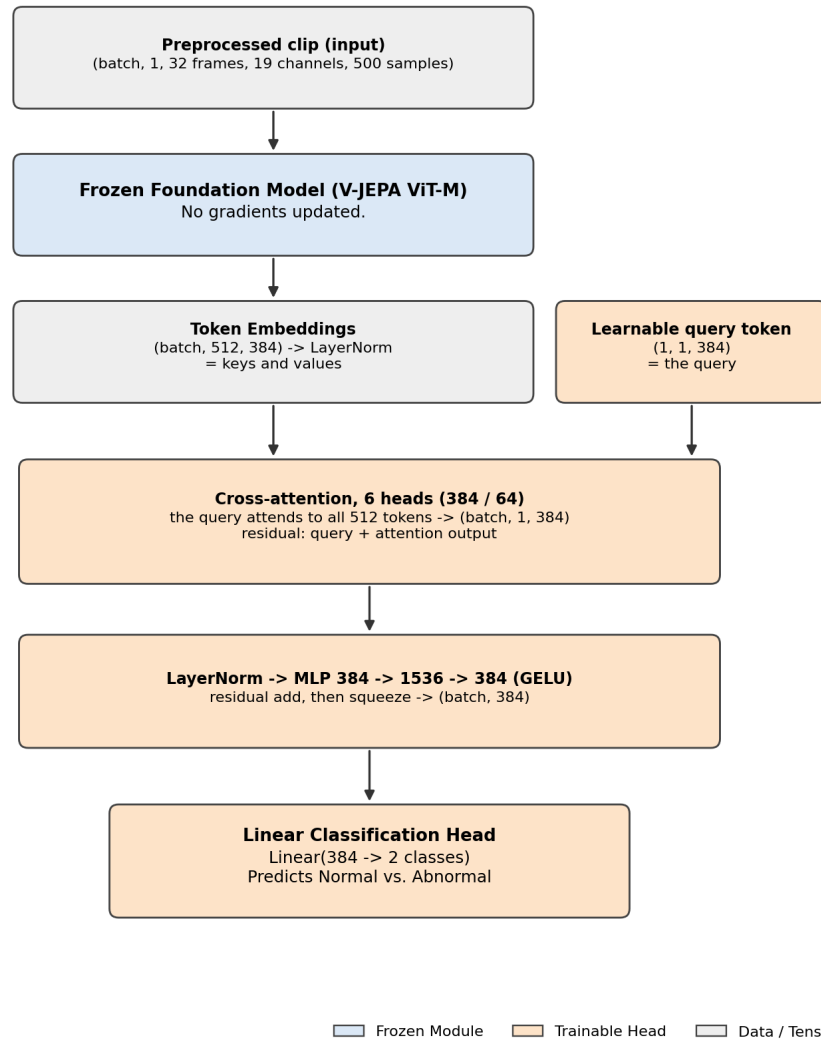


Figure 15: The paper’s Fig-3 Attentive Probe — a learnable query token attends over all 512 tokens via full multi-head cross-attention, followed by an MLP [1].

The Fig-3 Attentive Probe (‘AttentiveClassifier’) is the larger, frozen-only alternative: instead of one score per token, a single learnable query token attends over all 512 tokens through a full cross-attention block (6 heads of 384/64 for ViT-M), and the result passes through an MLP before the final classification layer. This gives it considerably more trainable weights than the §4.2 head, which is why it is the more forgiving of the two frozen heads for the released checkpoint, and why, in Experiment 4 below, we also try our own even simpler linear probe on the same frozen features, to check that neither of the authors’ heads is itself the bottleneck [1].

Table 6: The three evaluation heads used throughout this report, with exact trainable-parameter counts. The Linear Probe is a scikit-learn LogisticRegression, not a neural network, so its “parameters” are fitted coefficients rather than gradient-trained weights; the other two are PyTorch modules from the authors’ own upstream/eeg-vjepa code [1].

Head	Sits on	Trainable parameters	Used with
Linear Probe	ViT-M mean+std (768-dim)	769 (sklearn coefficients + intercept)	Released/random ViT-M (Experiments 1, 4, 6)
Linear Probe	FEI alone (256-dim)	257	FEI pre-training ablations
Linear Probe	FEI+GTJ concat (512-dim)	513	Combined FEI+GTJ runs (Experiment 6)

Head	Sits on	Trainable parameters	Used with
§4.2 AttentionClassifier	ViT-M tokens (384-dim)	1,155	Released/random ViT-M, frozen and fine-tuned (Experiments 1, 2)
Fig-3 AttentiveClassifier	ViT-M tokens (384-dim)	1,775,618 (1.78M)	Released/random ViT-M, frozen only (Experiment 1)

Table 7: Experiment 1 — released-checkpoint TUAB AUROC/BACC/AUC-PR (final epoch; BACC and AUC-PR at the default 0.5 threshold, the same convention used everywhere in this report) vs. random-init controls and the paper's reported AUROC (*best-epoch is optimistic, chosen by looking at the test set, and shown only for reference; ViT-B was run without a random-init twin) [1]. Runs: one evaluation head trained per TUAB training subset (s0–s4); each random-init run (s0–s2) also draws its own random weights, seeded with its subset number.

Encoder + head	Runs	AUROC	BACC	AUC-PR	Best-ep. AUROC*	Paper's AUROC
ViT-M, §4.2 head (paper's own)	5	0.671 ± 0.020	0.626 ± 0.023	0.685 ± 0.021	0.737	0.877
ViT-M, Fig-3 head	5	0.713 ± 0.025	0.659 ± 0.033	0.717 ± 0.035	0.784	0.877
Random-init ViT-M, §4.2 head	3	0.787 ± 0.006	0.704 ± 0.004	0.792 ± 0.010	0.844	—
Random-init ViT-M, Fig-3 head	3	0.760 ± 0.016	0.707 ± 0.022	0.756 ± 0.006	0.839	—
ViT-B, §4.2 head	5	0.662 ± 0.030	0.598 ± 0.020	0.654 ± 0.024	0.715	0.879
ViT-B, Fig-3 head	5	0.762 ± 0.031	0.687 ± 0.029	0.753 ± 0.032	0.814	0.879

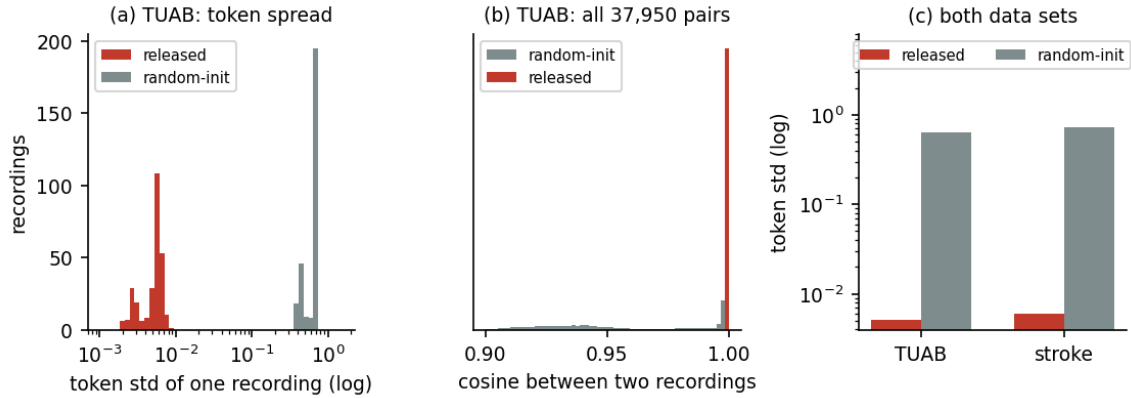


Figure 16: Direct evidence of representation collapse — the released checkpoint's per-recording output spread (token std) is about 125× smaller than a random-weight twin's, on both TUAB and the (never-seen-in-pretraining) stroke dataset, and its recordings are almost perfectly cosine-similar to one another (mean 0.999982) regardless of diagnosis. Token spread is the deciding measure; random weights also reach a cosine of 0.9991 on the stroke cohort.

Every released-checkpoint configuration falls roughly 0.1–0.2 AUROC short of the paper. More strikingly, a random, untrained ViT-M beats the released, pre-trained one: by 0.116 AUROC with the §4.2 head, 0.046 with the Fig-3 head (one-sided Mann–Whitney over 5 released and 3 random-weight runs; $p = 0.018$, the smallest these counts allow, and 0.036; two-sided, 0.036 and 0.071, so the Fig-3 gap alone would not pass a two-sided test). Training curves over all 500 epochs of the authors' own recipe show this gap from the very first epochs. More training wasn't going to fix it. Experiment 3 shows why: the released encoder's output barely varies from one recording to the next, so a classifier on top of it has almost nothing to work with.

Experiment 2 — Fine-tuning vs. training from scratch.

If the released weights are collapsed, does starting from them still help once the whole network is fine-tuned end-to-end, rather than kept frozen? The paper reports 0.885 AUROC after full fine-tuning. We fine-tuned the whole ViT-M plus §4.2 head twice (once from the released weights, once from random weights) on the same five training subsets, using the authors' own recipe (500 epochs, batch size 2, learning rate 1e-3, cosine decay). Batch 2 looked worth increasing for speed, so we tried 256 once on a protocol check before committing: it under-trains badly enough (59.7 AUROC) that the slower batch 2 won without a second thought. A “training subset” here, and in every TUAB experiment below, is one of five separate samples drawn from TUAB's 2,717-recording training pool (2,993 total minus the 276-recording eval set from Experiment 1), mirroring the paper's own five-subset protocol, whose exact recording membership the authors never published (§5.4).

Table 8: Experiment 2 — final-epoch AUROC/BACC/AUC-PR (mean $\pm$ sd over 5 training subsets), fine-tuning from the released checkpoint vs. from random weights, on identical training subsets (paper's own fine-tune recipe).

Start point	AUROC	BACC	AUC-PR
Fine-tune (released weights)	0.755 $\pm$ 0.053	0.684 $\pm$ 0.043	0.763 $\pm$ 0.055
From scratch (random weights)	0.768 $\pm$ 0.114	0.699 $\pm$ 0.077	0.795 $\pm$ 0.099

The two starting points are not significantly different (paired t-test and Wilcoxon both $p = 0.81$; any two of the five subsets share 98–134 of their 546 recordings, which both tests ignore, so allowing for that overlap would widen the uncertainty, not narrow it). From-scratch training even wins on 3 of 5 subsets. Starting from the released, “pre-trained” weights gives no measurable advantage over starting from nothing, which is exactly what we'd expect if those weights carry no usable information.

Experiment 3 — Confirming representation collapse.

Experiments 1 and 2 both point the same way, so we tested it directly. For every recording, Maruf measured token standard deviation (how much the encoder's output varies within one recording) and the mean cosine similarity between every pair of recordings' embeddings, where 1.0 would mean every recording produces an identical output. Operationally (code/tuab_collapse_check.py), each recording contributes one deterministic clip (32 frames spread over its first 96), which the encoder turns into 512 tokens of 384 dimensions; token std is the standard deviation over those 512 tokens, taken per dimension and averaged over the 384 dimensions, and the cosine is computed between recordings' mean-pooled 384-dimensional embeddings and averaged over every pair of recordings.

Table 9: Experiment 3 — collapse diagnostics on two completely different datasets, one of which (stroke) the released model never saw in any form during pre-training.

Dataset	Encoder	Token std (mean)	Mean cosine between recordings
TUAB eval (276 recs)	Released ViT-M	0.0051	0.999982
TUAB eval (276 recs)	Random-init ViT-M (3 seeds)	0.634	0.974
Stroke (15 subjects)	Released ViT-M	0.0060	0.999994
Stroke (15 subjects)	Random-init ViT-M (3 seeds)	0.717	0.9991

The two groups don't overlap at all. The largest token std measured for the released checkpoint, on any recording, was 0.0085. The smallest measured for a random-weight twin was 0.379. That rules out measurement noise. The released EEG-VJEPA checkpoints give almost the same output for every input, on two entirely different patient populations. No preprocessing variation we tested changes this, and it holds on both datasets, so it appears to be a property of the released weights rather than of our pipeline or of which dataset we test on. We even checked whether our own z-scoring convention could be doing this: per-recording and per-window normalisation give the same collapsed number (token std 0.0059, cosine 1.0000) on the same 120 recordings. We fixed no numeric collapse threshold in advance: the criterion is separation from every

random-weight twin, and a gap of 0.0085 against 0.379 needs no threshold to read. Token spread is the deciding measure. Cosine alone is not: on the 15-subject stroke cohort even random weights give a cross-recording cosine of 0.9991 (Table 9).

That raised an obvious question: why would collapse even be possible, given that the architecture has an anti-collapse safeguard built in? Their public implementation includes a variance-hinge term, ``relu(1 - std)``, that penalises the predictor's output for having too little spread across the masked patches of a clip — the same kind of safeguard that motivated FEI's own BYOL-style design. Maruf went looking in their published training configs and found a likely explanation: the exact config file that produced the released ViT-M checkpoint (``configs/pretrain/cluster_vitl16_EEG.yaml``) sets that term's weight, ``reg_coeff``, to **0.0**. Switched off. What's left is a plain, unnormalised L1 loss between the predictor's output and a layer-normalised target, and a plain L1 loss can be satisfied almost as cheaply by a near-constant output as by a genuinely informative one. We have not re-run their pre-training with this term switched back on, so this is the most likely explanation we found in their own code and config, not an independently re-verified causal test. What is demonstrated is the collapse itself (Table 9) and the setting in the released config; that the setting caused the collapse is a hypothesis. A second, also untested, candidate sits in the same code: pre-training saves ``jepa-best``, the checkpoint with the lowest training loss, and the released file is that checkpoint (epoch 353). Under a loss that a near-constant output can satisfy, the lowest-loss checkpoint may well be the most collapsed one.

Experiment 4 — The simplest possible probe, plus a classical baseline.

Maybe the released features still carry some useful signal that the paper's own classifier heads just fail to extract. We tried the simplest alternative available: mean and standard deviation of the encoder's token outputs (768 numbers total), fed to a StandardScaler plus logistic regression, and, for reference, a purely classical, hand-crafted relative band-power feature set with no learned encoder at all.

Table 10: Experiment 4 — AUROC (a simple linear probe recovers more signal than the paper's own head, but still falls short of the paper's 0.877 and is not clearly better than random weights) plus the full operating point at the full-training-set size (BAcc/AUC-PR/Sens/Spec at the default 0.5 threshold; random-init ViT-M has no single full-train run, hence the dashes).

Features	AUROC, subsets s0–s4	AUROC, full-train	BAcc, full-train	AUC-PR, full-train	Sens, full-train	Spec, full-train
Band power (hand-crafted, no encoder)	0.730 ± 0.014	0.757	0.670	0.718	0.754	0.587
Random-init ViT-M	0.784 ± 0.023	—	—	—	—	—
Released ViT-M	0.805 ± 0.013	0.838 [0.787, 0.884]	0.762	0.846	0.730	0.793

This simpler probe does recover more signal than the paper's own heads (0.805 ± 0.013 vs. 0.671–0.713 for the paper's heads on the same five training subsets; 0.838 on the full training set), and both ViT-M variants beat the classical band-power baseline. But the released checkpoint still can't be told apart from a random-weight twin with any confidence (one-sided Mann–Whitney $p = 0.196$, 5 released against 3 random-weight runs), and its full-training-set score of 0.838 still sits 0.039 below the paper's reported 0.877. Of the three frozen heads we ran, this linear probe gives the released checkpoint its highest TUAB eval AUROC; we picked it after seeing all three results, which can only flatter the released model, and it is the bar Experiment 6 compares FEI+GTJ against.

Read together with Experiment 1's per-head comparison (Table 7), this is the ablation sweep over everything that sits between a frozen embedding and a final decision: encoder (band power / random-init ViT-M / released ViT-M) crossed with head (Linear Probe / §4.2 AttentionClassifier / Fig-3 AttentiveClassifier). Swapping either one in isolation does move the number, but neither swap closes the gap to the paper's reported 0.877 — the shortfall behaves as a property of the released weights rather than of which head reads them.

One caveat on the band-power baseline carries through Experiment 6: the 0.730–0.757 numbers above use relative spectral power only, which discards each channel's absolute signal amplitude. We ran a separate sanity check, mainly to confirm our relative-power implementation wasn't itself buggy rather than to chase the strongest possible classical feature, and found that adding absolute (log) power on a longer 20-minute window

(not this report's 5-minute one) raises the baseline to 0.808: narrowing, without closing, the gap to FEI+GTJ and the released ViT-M's linear-probe score. We kept the weaker, 5-minute relative-power version in every TUAB experiment so they stay comparable; Experiment 5 instead gets its own NMT band-power arm, scored on the same folds as the encoders it is compared with. Read “beats band power” as beating this specific, deliberately simple baseline, rather than the strongest classical feature set we found.

Experiment 5 — FEI+GTJ validated on NMT, the data it was trained on.

With the released model ruled out as a usable pre-trained encoder, we evaluated FEI+GTJ on subject-independent, seed-0, five-fold cross-validation across all 2,417 NMT recordings. Each fold's encoders are pre-trained only on that fold's own training data — no test recording, labelled or not, is ever seen during pre-training. The folds are scikit-learn's StratifiedKFold (5 splits, shuffled, random_state = 0) over NMT's train and eval recordings concatenated in that order and stratified by label; NMT holds one recording per participant, so the split is also subject-independent. docs/act0_repro/nmt_folds_seed0.csv lists every recording's fold.

Table 11: Experiment 5 — FEI, GTJ and FEI+GTJ on NMT against the classical band-power baseline, seed-0 5-fold cross-validation, mean $\pm$ sd over folds, full metric set at the default 0.5 threshold. Band power is scored on the identical folds with the identical probe (code/act0_nmt_bandpower.py), on NMT's own 200 Hz input. Pre-trained rows are the original pre-training run; each rand- row is its random-init twin, averaged over 5 random seeds per fold (code/act0_nmt_rand_controls.py).

Features	AUROC mean $\pm$ sd (5- fold CV)	BAcc	AUC-PR	F1	Sens	Spec
Band power (classical, NMT)	0.786 $\pm$ 0.009	0.720 $\pm$ 0.013	0.429 $\pm$ 0.033	0.458 $\pm$ 0.013	0.773 $\pm$ 0.025	0.667 $\pm$ 0.010
FEI alone	0.819 $\pm$ 0.029	0.729 $\pm$ 0.025	0.563 $\pm$ 0.048	0.486 $\pm$ 0.028	0.708 $\pm$ 0.049	0.750 $\pm$ 0.019
rand-FEI (random-init control)	0.810 $\pm$ 0.025	0.728 $\pm$ 0.019	0.548 $\pm$ 0.054	0.494 $\pm$ 0.025	0.675 $\pm$ 0.032	0.780 $\pm$ 0.018
GTJ alone	0.836 $\pm$ 0.021	0.752 $\pm$ 0.033	0.572 $\pm$ 0.041	0.522 $\pm$ 0.036	0.718 $\pm$ 0.064	0.786 $\pm$ 0.017
rand-GTJ (random-init control)	0.794 $\pm$ 0.009	0.717 $\pm$ 0.014	0.483 $\pm$ 0.018	0.481 $\pm$ 0.015	0.658 $\pm$ 0.032	0.777 $\pm$ 0.010
FEI+GTJ (fused)	0.865 $\pm$ 0.024	0.778 $\pm$ 0.030	0.657 $\pm$ 0.062	0.570 $\pm$ 0.034	0.725 $\pm$ 0.060	0.831 $\pm$ 0.011
rand- FEI+GTJ (random-init control)	0.834 $\pm$ 0.007	0.749 $\pm$ 0.010	0.586 $\pm$ 0.033	0.542 $\pm$ 0.014	0.658 $\pm$ 0.020	0.840 $\pm$ 0.012

Fusion beats FEI alone and GTJ alone in every one of the five folds, a strong sign that the two branches pick up different aspects of the signal. The five training sets overlap (any two share three quarters of their recordings), so an ordinary paired t-test over folds understates the variance; every p-value in this experiment is therefore the Nadeau–Bengio corrected resampled t-test [20], Holm-adjusted [21] over the eleven comparisons of Table 20 (Appendix §11.3). The pre-trained arms there are averaged over three independent pre-training runs, the original plus two re-runs with new seeds (code/act0_nmt_pretrain_replicate.py), whose fused means were 0.865, 0.865 and 0.852; the runs are averaged within each fold before testing, so run-to-run variation is folded into the fold means rather than modelled separately. On those means fusion is ahead of FEI by +0.041 ($p = 0.027$) and of GTJ by +0.030 ($p = 0.012$ before Holm, 0.082 after). Against band power on the same folds, the fused model is ahead by +0.075 AUROC (all 5 folds, $p = 0.029$) and GTJ by +0.045 (all 5 folds, $p = 0.027$); FEI's +0.034 (4 of 5 folds) is not significant ($p = 0.45$). The random-init twins change how that margin should be read. An untrained fused encoder already reaches 0.834 ± 0.007 , itself ahead of band power by +0.048 (all 5 folds, $p = 0.026$), so most of the fused model's advantage comes from its architecture and input representation. Pre-training adds +0.027 on top (three-run mean 0.861 against 0.834), better in all 5 folds but not significant once corrected ($p = 0.15$ before Holm, 0.45 after); the gain is +0.036 for GTJ (all 5 folds, $p =$

0.019 before Holm, 0.12 after) and only +0.010 for FEI (3 of 5 folds). This band-power arm is the same relative-band-power feature as Table 10's, recomputed on NMT's own 200 Hz input with its bands running from 0.5 to 45 Hz (code/baseline_bandpower.py).

Experiment 6 — FEI+GTJ on a different hospital's data (TUAB): NMT-only and adapted.

FEI+GTJ was built and validated entirely on NMT (Pakistan). Experiment 6 tests it on TUAB (USA): a different hospital, different hardware, a different patient population. We separate two settings. NMT-only transfer: the encoder is pre-trained on NMT only and never sees a TUAB recording, labelled or not, before its frozen features are probed. This is sometimes called zero-shot, but only the encoder's pre-training is: the linear classifier on top is trained on labelled TUAB recordings, so it tests how the representation transfers across datasets, not zero-shot classification. Target-domain unlabelled adaptation: the encoder is re-pre-trained on NMT plus TUAB's unlabelled training recordings (never the eval set, never a label) and then probed the same way. In both settings the linear probe is fitted on TUAB's labelled training subsets and scored on the 276-recording eval set, exactly as in Experiment 4.

Table 12 reports both settings. FEI's NMT-only row is the clean cross-dataset test of representation transfer. GTJ does not survive the NMT-only transfer: it falls below its own random-init twin (0.840 vs 0.863 full-train AUROC), consistent with TUAB's differently filtered spectrum being out of distribution for an encoder trained on NMT's. The adapted rows therefore re-pre-train GTJ, and the FEI inside the adapted fused model, on NMT plus TUAB's unlabelled training recordings. The NMT-only FEI and the FEI inside the adapted fused model are two separately pre-trained encoders, rather than the same weights reused. An untrained encoder is the same in both settings, so each random-init row serves as the control for both.

Table 12: Experiment 6 — frozen FEI, GTJ and FEI+GTJ on TUAB. NMT-only rows: encoders pre-trained on NMT alone; adapted rows on NMT plus TUAB's unlabelled training recordings. Matched random-init controls for every arm; full-train operating point (BAcc/AUC-PR/Sens/Spec at the default 0.5 threshold) alongside AUROC.

Features	AUROC subsets s0–s4	AUROC full-train	BAcc full-train	AUC-PR full-train	Sens full-train	Spec full-train
Band power (classical)	0.730 ± 0.014	0.757	0.670	0.718	0.754	0.587
Released ViT-M (linear probe, Table 10)	0.805 ± 0.013	0.838	0.762	0.846	0.730	0.793
FEI (NMT-only)	0.835 ± 0.015	0.872	0.785	0.871	0.730	0.840
rand-FEI (random-init control)	0.790 ± 0.020	0.813	0.743	0.829	0.698	0.787
GTJ (NMT-only)	0.805 ± 0.005	0.840	0.768	0.835	0.722	0.813
GTJ (NMT+TUAB -train, adapted)	0.811 ± 0.012	0.858	0.767	0.853	0.714	0.820
rand-GTJ (random-init control)	0.819 ± 0.014	0.863	0.779	0.848	0.738	0.820
FEI+GTJ (NMT-only)	0.821 ± 0.006	0.857	0.792	0.852	0.730	0.853
FEI+GTJ (NMT+TUAB -train, adapted)	0.821 ± 0.012	0.862	0.764	0.855	0.722	0.807
rand-FEI+GTJ (random-init control)	0.834 ± 0.014	0.861	0.761	0.860	0.722	0.800

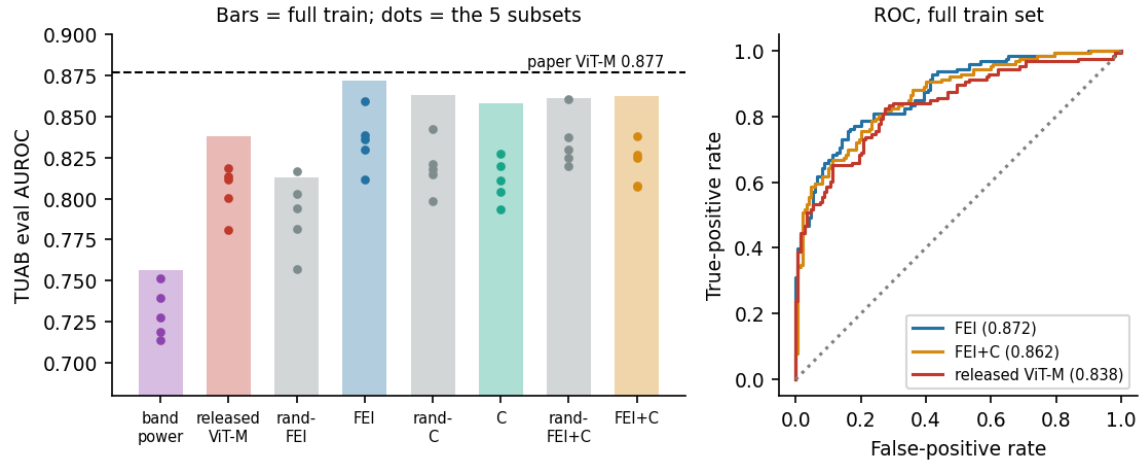


Figure 17: FEI+GTJ's TUAB transfer alongside the released ViT-M, band power, and every random-init control — dots are the five training subsets, bars are the full 2,717-recording training set.

Adapted, FEI+GTJ scores above the released ViT-M and clearly beats band power (0.862 vs. 0.838 vs. 0.757 on the full training set), though the gap between FEI+GTJ and ViT-M, +0.024, isn't statistically significant ($p = 0.147$). Two seeded re-runs of that adapted pre-training (code/tuab_feic_joint_reps.py, scored the same way) reach 0.851 and 0.855; resampling the three runs and the test recordings together, FEI+GTJ's lead over ViT-M is +0.018 (95% CI $[-0.028, 0.064]$, $p = 0.220$), still not significant. NMT-only, the fused model reaches 0.857 on the full training set (0.821 ± 0.006 over the five subsets), so most of the adapted model's score does not depend on seeing TUAB during pre-training.

FEI's NMT-only pre-training gain, though, is statistically significant on its own: FEI beats its random-init twin by +0.059 AUROC (95% CI $[0.019, 0.097]$, $p = 0.002$): so what FEI learned on NMT alone carries information about abnormal EEG on TUAB that its architecture by itself does not supply (one dataset pair). That p -value resamples the 276 test recordings for one set of trained encoders (five, one per NMT training fold, ensembled), so it says nothing about how the gain varies between pre-training runs. We therefore repeated the NMT pre-training and scored each run on TUAB the same way (code/tuab_fei_reps.py), against the same random-init twins (the same script gives the original run's +0.059 again): Experiment 5's two seeded re-runs, plus two further FEI-only runs with the same recipe and seeding, trained for this check alone (Experiment 5's analysis was fixed at three runs of both branches before these two were trained, and with no GTJ partner they cannot enter its fused comparison). Across the five runs the gain over the twin averages +0.048, ranging from +0.036 to +0.059; of the 25 individual fold encoders, 23 score above the random-init ensemble (0.813) and 24 above the best single random-init encoder (0.809). A bootstrap that resamples the five pre-training runs, the twin's three random-init seeds and the 276 test recordings together, so that all three sources of uncertainty enter one interval (Table 24), gives a mean gain of +0.048 (95% CI $[0.015, 0.108]$, $p = 0.0055$). All five runs are scored on the same test set, so they show that the gain does not hinge on one pre-training seed; they are not five independent replications of the transfer, and with five runs and three seeds the between-run part of that interval is itself rough.

GTJ is the more sobering half of the same table. Adapted GTJ vs. its random twin is -0.005 ($p = 0.642$, not significant), adapted FEI+GTJ vs. its random twin is only +0.001 ($p = 0.455$), and NMT-only GTJ is below its twin outright (0.840 vs 0.863). Two seeded re-runs of the adapted pre-training tell the same story: GTJ -0.017 ($p = 0.879$) and -0.023 ($p = 0.944$), FEI+GTJ -0.010 ($p = 0.734$) and -0.006 ($p = 0.665$) against the same twins, and the bootstrap over the three runs, the twins' random-init seeds and the test recordings gives -0.015 (95% CI $[-0.040, 0.031]$, $p = 0.671$) for GTJ and -0.005 (95% CI $[-0.033, 0.043]$, $p = 0.431$) for FEI+GTJ. Neither adapted GTJ nor adapted FEI+GTJ shows a pre-training gain in any of the three runs: consistent across three initialisations, which is not proof that none exists. FEI on its own, from the same adapted pre-training runs, stays above its random-init twin in all three (AUROC 0.840–0.859): +0.046 ($p = 0.0085$), +0.027 ($p = 0.113$) and +0.041 ($p = 0.020$). The same bootstrap over runs, random-init seeds and test recordings gives +0.038 (95% CI $[-0.0003, 0.104]$, $p = 0.026$; the interval only just reaches zero), a little below the NMT-only runs' +0.048. That gain does not show in the fused comparison: the fused model's twin contains random-init GTJ, which on its own already reaches 0.863, above all three adapted FEI runs. The same collapse diagnostics as Experiment 3 show this isn't collapse in disguise: on the TUAB test recordings the adapted GTJ encoders of

all three runs keep an embedding std of 0.33–0.39 (cosine 0.59–0.71), and every FEI encoder, NMT-only or adapted, re-runs included, 0.45–1.08 (0.64–0.84). NMT-only GTJ is the one compressed case (std 0.04–0.07, cosine 0.96–0.99, below every random-init twin's 0.11), far less so than the released checkpoint (Experiment 3) and in line with the spectrum shift described above. GTJ's spectrogram input and architecture are doing useful work on TUAB: the pre-training itself just isn't the reason why, a distinction the failure-axis analysis below returns to.

Visual check — do t-SNE / UMAP maps show separate normal/abnormal clusters?

Our original proposal promised a 2-D visual check of the learned features, in the spirit of the paper's own UMAP figure, which shows visibly separated clusters. We made the same kind of map for the released ViT-M, its random-init twin, FEI+GTJ, and its random-init twin, on the TUAB evaluation set.

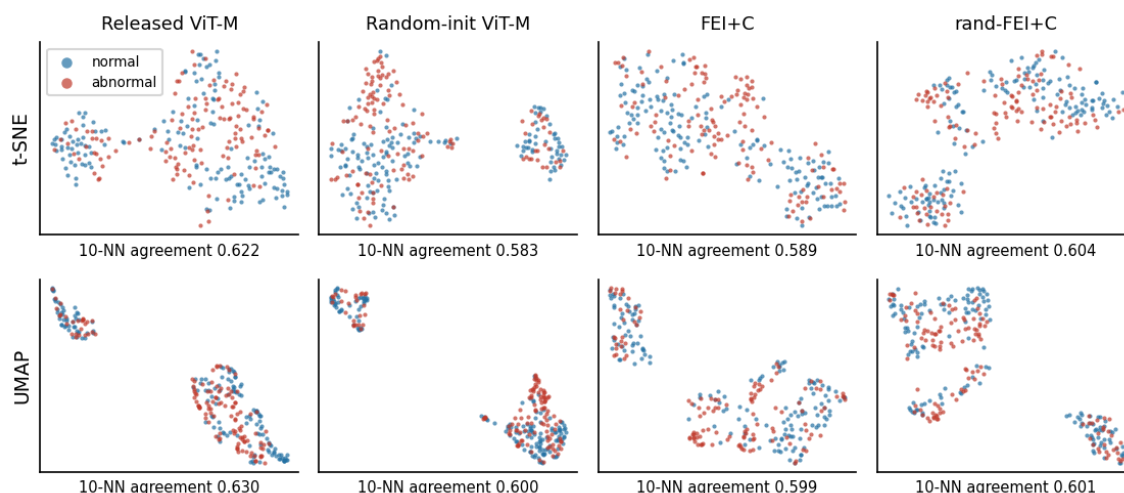


Figure 18: t-SNE (top) and UMAP (bottom) maps of four feature sets on TUAB — blue = normal, red = abnormal. No map, learned or random, shows the two labels forming separate regions.

None of the four maps, including the two random-weight ones, show normal and abnormal recordings forming separate regions. The 10-nearest-neighbour label-agreement statistic — the fraction of a recording's 10 nearest neighbours that share its label — stays in a narrow 0.598–0.630 band for all four feature sets, on the original high-dimensional features as well as on their 2-D t-SNE/UMAP projections (released ViT-M: 0.623 original, 0.622 t-SNE, 0.630 UMAP; its random-init twin: 0.600, 0.583, 0.600; FEI+GTJ: 0.629, 0.589, 0.599; its random-init twin: 0.598, 0.604, 0.601), barely above the 0.502 chance level, even though a linear probe on the same features reaches 0.838 (released ViT-M) or 0.862 (FEI+GTJ) AUROC. These encoders seem to separate the two classes along a specific direction in embedding space that a linear classifier can find, rather than through an overall 2-D layout that t-SNE or UMAP would reveal. The released ViT-M's map even looks just as structured as the healthy random-init one, a useful reminder that a nice-looking visualisation isn't evidence against collapse on its own. Only the direct diagnostics in Experiment 3 caught that.

Experiment 7 — Pipeline-soundness gate and the clinical BSI baseline (stroke).

Before trusting any model's number on the small, 15-subject stroke cohort, Soumya first checked that the data pipeline reproduces a well-established classical effect: stroke patients show higher left-right power asymmetry (a higher Brain Symmetry Index) than healthy controls. This gate was pre-declared, with a fixed significance threshold, before anyone looked at a single model's classification result (it is the gate section of runs/act0/phaseB/PREREGISTRATION.md, written on 2026-09-25 at 23:55, UTC+6; Appendix §11.2).

Concretely, Soumya implemented the *pairwise-derived* BSI (pdBSI) of Sheorajpanday et al. (2009) [22], a more localized variant than the single whole-head ratio of the original BSI cited above (van Putten & Tavy, 2004 [9]): for each of six symmetric electrode pairs (FC1/FC2, FC3/FC4, C1/C2, C3/C4, C5/C6, CP1/CP2) and every frequency bin in the band of interest, $\text{pdBSI} = \text{mean}(|(P_R - P_L) / (P_R + P_L)|)$, where P_L and P_R are that pair's left- and right-hemisphere spectral power; the reported score averages this ratio across all six pairs and every bin. Taking the mean of six local ratios rather than one global ratio makes the measure more sensitive to a focal asymmetry — expected after a stroke restricted to one hemisphere — which is why it was the better choice for a cohort this small.

Table 13: Experiment 7(a) — the pipeline-soundness gate and the classical spectral features around it. The pre-declared gate is the first row (broadband BSI higher in stroke, two-sided Mann–Whitney $p < 0.05$): passed, $p = 0.003$. The pre-registration also asked for the direction of DAR, DTABR and relative delta, alpha and beta power, and all five agree; the remaining rows are reported for completeness. Every feature moves in the expected post-stroke direction in the primary cohort, and 10 of 11 reach $p < 0.05$ there (the exception is BSI theta, $p = 0.113$). The sensitivity cohort — built from a shorter window that includes some task-adjacent data, by its own definition — confirms every BSI feature but loses significance on the relative-power and ratio features, consistent with a noisier comparison rather than a reversed effect.

Feature	Direction in stroke	Primary median: stroke / control	Primary p	Sensitivity p
BSI 1–30 Hz (broadband)	↑	0.149 / 0.073	0.003	< 0.001
BSI delta	↑	0.109 / 0.081	0.005	< 0.001
BSI theta	↑	0.098 / 0.062	0.113	0.021
BSI alpha	↑	0.188 / 0.091	0.012	0.002
BSI beta	↑	0.142 / 0.066	0.003	< 0.001
Relative delta power	↑	0.485 / 0.214	0.008	0.068
Relative theta power	↑	0.172 / 0.106	0.036	0.016
Relative alpha power	↓	0.119 / 0.264	0.050	0.173
Relative beta power	↓	0.133 / 0.378	0.026	0.068
DAR (delta/alpha power ratio)	↑	3.407 / 0.957	0.018	0.122
DTABR ((delta+theta)/(alpha+beta))	↑	1.758 / 0.517	0.002	0.027

Table 14: Experiment 7(b) — the Brain Symmetry Index alone, with no learned encoder at all, is already a strong and stable stroke-vs-control classifier under leave-one-subject-out CV.

Cohort	BSI classifier, LOSO AUROC	95% CI	Permutation p	Sens.	Spec.
Primary	0.907	[0.679, 1.000]	0.005	0.889	1.000
Sensitivity	0.938	[0.775, 1.000]	0.002	0.900	1.000

Both results are a sanity check that the pipeline recovers the expected BSI effect, and they make BSI a demanding reference baseline: alone it reaches 0.907 AUROC, and any self-supervised encoder here is judged against it, however interesting its pre-training.

Experiment 8 — FEI+GTJ on stroke, after fixing a distribution-shift problem.

The first attempt to run FEI+GTJ on stroke data ran straight into a wrong assumption. We'd treated NMT and the stroke cohort as if they shared the same recording band, 0.5–40 Hz. They don't: NMT's own EDF header has a blank prefilter field, and its real spectrum rolls off closer to 35 Hz, while the stroke side's 0.5–40 Hz filter passes content up to about 44–45 Hz. Soumya traced this down after GTJ's BatchNorm layers, which store NMT's input statistics, saw the stroke cohort's spectrum as wildly out of range (z-scores of 4.3 standard deviations or more) and collapsed. Same failure mode as Experiment 3, this time self-inflicted by a spectrum mismatch rather than a training-objective problem.

The fix: a label-free, per-channel zero-phase filter that maps the stroke cohort's mean spectrum onto NMT's, gated on numerical thresholds. This step was not pre-registered: under the pre-registered input, FEI+GTJ had scored 0.833 LOSO AUROC, below its random-init twin's 0.889, and that result is what prompted the fix. The recipe, its gates and the comparisons to run were written to runs/act0/phaseB_clean/POSTHOC.md before any LOSO run on the corrected input (Appendix §11.2), and every gate passed. The largest gap between the stroke and NMT log-spectra fell from 5.81 to 0.18 decades (gate: ≤ 0.25 decades), GTJ's BatchNorm inputs fell from $|z| \approx 4.3$ to ≈ 0.37 , and no embedding dimension stayed constant across subjects. Experiment 8 is therefore a post-hoc repair of a distribution shift, tested on the same 15 subjects whose result prompted it, not an

independent confirmation. Refitting the filter inside each fold (Table 19) rules out test-subject leakage, but the method itself was shaped by this cohort; whether it carries over needs a new cohort.

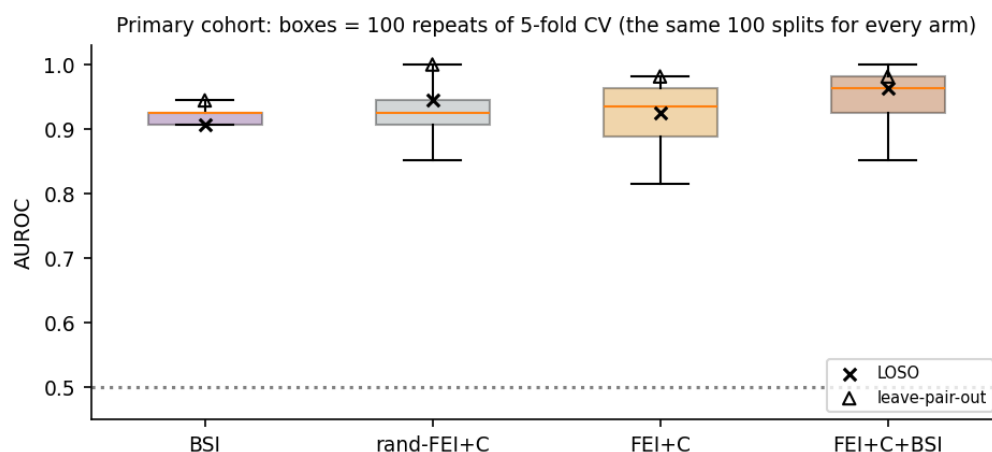


Figure 19: Experiment 8 — BSI, FEI+GTJ, its random-init twin, and BSI+FEI+GTJ combined, under 100 repeats of 5-fold cross-validation on the primary stroke cohort ($\times$ = leave-one-subject-out, Δ = leave-pair-out, an even stricter test that holds out one stroke case and one control case at a time).

Table 15: Experiment 8 — FEI+GTJ on the spectrum-corrected stroke input, across three different cross-validation schemes for robustness.

Arm	LOSO AUROC	5-fold $\times 100$ (mean $\pm$ sd)	Leave-pair-out	Permutation p (LOSO)
BSI (clinical baseline)	0.907	0.920 $\pm$ 0.012	0.944	0.005
rand-FEI+GTJ (random-init control)	0.944	0.930 $\pm$ 0.037	1.000	0.004
FEI+GTJ	0.926	0.918 $\pm$ 0.051	0.981	0.003
FEI+GTJ + BSI (combined)	0.963	0.951 $\pm$ 0.039	0.981	—

FEI+GTJ reaches 0.926 LOSO AUROC against BSI's 0.907 (Δ = +0.019, 95% CI [-0.240, 0.280], p = 0.523): a similar point estimate, not significantly different either way under every cross-validation scheme we tried. We set no equivalence margin in advance, and an interval this wide cannot show equivalence either, so the cohort is simply too small to rank the two. At the default 0.5 threshold, FEI+GTJ, BSI alone, and FEI+GTJ+BSI combined all land on the identical confusion matrix — 8 of 9 stroke cases and all 6 controls called correctly, one stroke case missed (Acc 0.933, BAcc 0.944, F1 0.941, Sens 0.889, Spec 1.000) — differing only in AUROC, i.e. in how confidently each ranks that one missed case; the random-init twin is very slightly worse at this same threshold (one extra false positive: Acc 0.867, BAcc 0.861, F1 0.889, Spec 0.833). It does not show a significant advantage from pre-training as a whole on this task: FEI+GTJ vs. its own random-init twin is -0.019 LOSO AUROC (p = 0.826), not a significant difference in either direction. Splitting this by branch (exploratory, single-arm, not pre-registered) shows the same pattern as TUAB: FEI's pre-training clears its random twin in 100% of repeated 5-fold splits; GTJ's does not, only 16% of repeats. The single, deterministic LOSO split (every subject held out exactly once, so there is no fold-assignment randomness to repeat here) tells a more dramatic version of the same branch story: FEI alone reaches a perfect 1.000 LOSO AUROC — a threshold exists with zero errors, 9/9 stroke and 6/6 control — against GTJ alone's 0.833 and the random-init twins' 0.907 (FEI) / 0.852 (GTJ). A perfect score on 15 subjects is still fragile, since one atypically-recorded subject could flip it, which is why the repeated, randomized 5-fold win-rate above, not this single point estimate, is the statistic we'd stand behind. FEI+GTJ combined with BSI (0.963 LOSO) numerically beats BSI alone in 82% of repeated splits — promising, but not statistically significant at 15 subjects. With the spectral-matching filter refitted inside each fold instead (Table 19), FEI+GTJ stays at 0.926 and FEI+GTJ+BSI drops to 0.944, the same as its random-init twin. Experiment 5's two seeded pre-training re-runs went through the same frozen pipeline: the same spectrally matched input, nothing re-fitted, and the same encoder gates (code/stroke_phaseB_reps.py; bootstrap comparisons only, no permutation tests). FEI+GTJ scores 0.963 and 0.981 LOSO AUROC (original run 0.926; random-init twin 0.944, BSI 0.907), FEI alone

1.000 and 0.981 (twin 0.907) and GTJ alone 0.833 and 0.833 (twin 0.852); FEI's re-runs beat their twin in 100% and 96% of the repeated 5-fold splits, GTJ's in 16% and 15%. Against BSI, FEI+GTJ's re-runs differ by +0.056 ($p = 0.428$) and +0.074 ($p = 0.389$), neither significant. The spread across the three runs is pre-training-seed variation only: all three are scored on the same 15 subjects, so they add no clinical sample and leave the subject-level intervals above as wide as they were. The re-run GTJ encoders pass the same gates as the original (BatchNorm $|z|$ at most 0.38, embedding spread 0.74–0.82 of their spread on NMT, no constant dimensions).

The released ViT-M (VJEPA) reaches 0.981 LOSO AUROC on this same cohort, higher than FEI+GTJ, FEI+GTJ+BSI, or BSI alone. But we don't read this as the released checkpoint working here. On this 15-subject cohort it is just as collapsed as on TUAB (Table 9: token std 0.0060, cross-recording cosine 0.999994; each of its output features varies across the 15 subjects by only 0.0014 on average, against about 0.018 for a random-weight twin), so a linear probe is separating cases on a vanishing residual that feature-scaling blows back up, a residual that could just as easily track a recording-condition difference between the stroke and control sessions as track stroke itself. Consistent with that, the same checkpoint's score drops from 0.981 on this cohort to 0.863 on the broader sensitivity cohort (a superset of these 15 subjects: 10 stroke vs. 8 controls, first 300 s rather than the 285 s rest block): the kind of instability a collapsed representation produces, rather than a genuine clinical signal.

5.3.4 Interpretation and Conclusions on Data

The released EEG-VJEPA checkpoints don't reproduce the paper's own result, because they're collapsed (Experiments 1–3). 500-epoch fine-tuning under the authors' own recipe did not fix that (Experiment 2), and the simplest possible probe can't rescue it either (Experiment 4). FEI+GTJ, built in response, beats a classical baseline on the data it was trained on, although an untrained twin already accounts for about two-thirds of that margin (Experiment 5); on a different hospital's data, FEI's NMT-only pre-training still gives a significant gain (+0.048 AUROC averaged over five pre-training runs, 95% CI [0.015, 0.108]), and the fused model, re-pre-trained with that hospital's unlabelled recordings, scores close to the released checkpoint (difference not significant) and stays ahead of a classical baseline (Experiment 6), and, after a post-hoc input fix, scores close to a strong clinical baseline on a third, structurally different task, too close for 15 subjects to separate (Experiments 7–8). A t-SNE/UMAP visual check, matching our original proposal's promised deliverable, shows that 2-D unsupervised layout is the wrong tool for seeing why any of these encoders work, or why the released one doesn't.

A second analysis pass, outside the original proposal, re-examined the per-recording predictions already saved from the experiments above, with no retraining. The question: when FEI and GTJ are wrong, are they wrong the same way, or differently, and does that explain where fusion helps and where it doesn't?

- Children under 18 make up 43% of NMT's recordings but 65% of its abnormal-labelled ones, and FEI flags **62% of healthy 0–6-year-olds** as abnormal: by far its single largest source of false alarms, even though its adult performance is its strongest (age alone predicts the label with AUROC 0.609 on NMT, and FEI's scores correlate with age within the normal class at $r = -0.42$). **FEI's main failure mode is age-dependent.** Our working, untested interpretation is that FEI's pretext task (predicting how its target encoder embeds a window once random frequency bins are removed) makes it sensitive to slow-wave activity that is a normal feature of a healthy developing brain in young children, rather than a sign of pathology. This pattern cannot be checked on TUAB, which has only 2 recordings under 18.

- **FEI is also weaker on adult women, though this is only a flag.** On NMT adults, FEI reaches 0.773 AUROC on women vs. 0.864 on men; the gap narrows for GTJ (0.832 vs. 0.852) and for FEI+GTJ (0.846 vs. 0.883). The same direction appears on TUAB (FEI 0.865 on women vs. 0.883 on men). This isn't simply the age effect above in disguise, since it persists within children alone too (FEI 0.756 female vs. 0.767 male), but we have not run a significance test or identified a mechanism, so we report this as an observation for future work rather than a finding.

- On NMT, FEI and GTJ's per-recording errors overlap only modestly (score correlation 0.60), and fusion repairs 321 of FEI's 622 errors while introducing just 21 new ones. With the NMT-only encoders on TUAB, the two branches' scores instead correlate at 0.84 and 44 of FEI's 58 errors are also GTJ errors, so fusion has

little left to fix: this NMT-only FEI+GTJ (0.857) falls slightly below FEI alone (0.872; paired difference -0.015 , 95% CI $[-0.047, +0.016]$, $p = 0.36$). Table 12's fused row (0.862) is the separately re-pre-trained joint version. The two branches also fail for different reasons: FEI's main failure mode is age (it flags 62% of healthy 0–6-year-olds on NMT as abnormal), while **GTJ's main failures turn out to be distribution-shift failures, rather than age failures**. Fusion, in short, only helps when its two inputs disagree, which explains both Experiment 5's strong fusion gain on NMT and Experiment 6's weaker one on TUAB.

- **GTJ is confidently wrong roughly twice as often as FEI is.** Looking only at each model's mistakes, FEI's tend to sit close to the 0.5 decision boundary (a clinically useful property: uncertain cases could be flagged for human review), while GTJ makes confident, large-margin mistakes about twice as often on NMT and more than seven times as often on TUAB. Because fusion inherits GTJ's confident errors, this is a calibration weakness specific to GTJ, separate from its overall ranking ability.

- **A single global decision threshold is a poor fit across ages.** At the standard 0.5 threshold, FEI+GTJ catches only 50–69% of abnormal recordings in adults while over-calling many healthy children's recordings as abnormal, because the classifier is trained on a population where most abnormal examples are children, so its decision boundary is implicitly tuned for them. The model's *ranking* (AUROC) is fine in adults; what fails is the single shared threshold.

- In an exploratory run, GTJ's NMT AUROC dropped in every one of 5 folds (-0.014) when pre-trained jointly on NMT plus unlabelled TUAB rather than NMT alone, and its embedding spread on NMT fell too, a sign that it reallocates representational capacity toward whichever domain it is shown, rather than learning more general structure. **Feeding GTJ more data during pre-training, in this setup, made it worse.** Consistent with this, GTJ's past-predicts-future pretext loss was observed to plateau within roughly 10 epochs, suggesting that for this architecture and this amount of data, the task is simply too easy to force deep, transferable learning.

- **One Fig-3-head number is not reproducible even by us, on the same seed.** Maruf re-ran the attention-head extraction behind the Fig-3 attentive probe (Figure 15) to double-check it before freezing the plots, and got 72.1 AUROC the first time and 68.7 on the exact re-run: same seed, same code, same checkpoint (Table 7 reports the re-run; with the first run its five-subset mean would read 0.720). The likely cause is GPU-level non-determinism in one of the attention kernels (cuDNN does not guarantee bit-identical output across runs unless you force deterministic algorithms, which we had not). We did not chase this down further; it is flagged here because it is a real gap in our own reproducibility as well, beyond the paper's. The §4.2 AttentionClassifier numbers are bit-identical across re-runs, and no conclusion in this report depends on the Fig-3 head's exact value.

The released EEG-VJEPa checkpoints are collapsed and fail to reproduce the paper; our token-spread diagnostic is repeatable, and the released training config gives a likely cause. FEI+GTJ beats our classical baseline on NMT, with pre-training adding a consistent but not significant $+0.027$ AUROC over an untrained twin (three-run mean 0.861 vs 0.834); on stroke, after a post-hoc input fix, its AUROC is close to BSI's, with far too few subjects to call them equivalent. Across hospitals, FEI's NMT-only pre-training still gives a significant gain ($+0.048$ AUROC averaged over five pre-training runs, 95% CI $[0.015, 0.108]$); the fused model, re-pre-trained with TUAB's unlabelled recordings, scores close to the released ViT-M there, with no significant difference. On stroke only FEI shows a pre-training gain, and no model here significantly beats BSI. Table 16 compares these outcomes with the original proposal.

Table 16: Act 0 — original proposal aims against what was actually achieved.

Proposal aim	Status	Evidence
Part 1: reproduce the paper's frozen TUAB result	Done faithfully, as far as the paper's text specifies; result not reproduced	43/48 files identical to upstream; 0.671–0.762 vs. paper 0.877–0.879
Explain the shortfall	Achieved	Collapse: token std 0.005 vs. 0.63 for random weights (Exp. 3)
Added: fine-tune the released ViT-M	Done; no gain from pre-training	0.755 ± 0.053 vs. 0.768 ± 0.114 from scratch, $p = 0.81$ (Exp. 2)

Proposal aim	Status	Evidence
Changed: FEI+GTJ as a working pre-trained encoder	Achieved; pre-training gain modest	original run 0.865 ± 0.024 on NMT (three-run mean 0.861) vs. band power 0.786 and untrained twin 0.834 (pre-training +0.027, not significant); on TUAB, FEI's NMT-only pre-training +0.048 averaged over five runs (95% CI [0.015, 0.108]; original run +0.059); fused, with TUAB's unlabelled recordings added, close to released ViT-M (not significant) and beats band power (Exp. 5–6)
Part 2: healthy vs. stroke, frozen encoders + BSI, LOSO	Done	Gate passed $p = 0.003$; BSI 0.907; FEI+GTJ 0.926 (seed re-runs 0.963, 0.981) after a post-hoc input fix, not separable from BSI (Exp. 7–8)
Deliverable: t-SNE / UMAP of learned features	Done	No class clusters in any map, learned or random (10-NN agreement ≈ 0.60 vs. chance 0.502)
Our own EEG-VJEPA pre-training from scratch	Dropped	Out of scope by team decision, 2026-09-26, in favour of the diagnosis-and-replacement work above

5.4 Limitations of Tools

- **Compute.** Everything in this report, pre-training and evaluation alike, ran on one shared consumer GPU (RTX 5060 Ti) behind a thermal guard that pauses or kills a job if the CPU gets too hot. That capped how many random-init seeds we could afford for some controls (3, not 10) and ruled out the much larger pre-training runs that the released EEG-VJEPA or models like CBraMod/LaBraM use. Appendix §11.5 lists the checks we did not run and why.
- **Cohort size.** The stroke cohort is 15 subjects. Every confidence interval there is wide, and some comparisons (FEI+GTJ vs. its random-init twin, for one) can't rule out even fairly large true differences in either direction.
- **Demographic confound in the stroke cohort.** Per the dataset's own record description [13], the stroke patients (55.9 ± 14.2 years, 6 male / 4 female) are older and more often male than the controls (46.3 ± 23.6 years, 1 male / 6 female); these figures describe the whole record, not exactly the 15 subjects we analyse. Encoders can read age and sex off EEG, so part of any stroke-vs-control score in Experiments 7–8 may just be reading those group differences back out. With 15 subjects there's no way to separate the two statistically: Soumya flagged this while building the cohort table, and we checked it couldn't be fixed by just re-splitting, since the imbalance is in who was recruited, not how we split them.
- **Data access.** Our copy of TUAB came from an unofficial mirror rather than the licensed channel (see Legal Issues, §5.5.3). That limits what we can responsibly do with those specific numbers.
- **Unpublished details in the reference paper.** The EEG-VJEPA paper doesn't say exactly which 546 recordings make up each of its five reported training subsets, which training epoch its headline numbers come from, or the exact fine-tuning recipe behind its 0.885 AUROC. We fixed our own choices with a seeded sampler (code/tuab_released_eval.py; the five 546-recording subsets are listed file by file in docs/act0_repro/tuab_subsets.csv), but there's no guarantee they match what the authors actually did.
- **Software environment.** Ordinary friction (recent Debian/Ubuntu Python blocks a direct `pip install` outside

a virtual environment, that kind of thing) cost real hours of environment setup that have nothing to do with EEG. It was a genuine time sink, and none of it will ever show up in a related-work section. The environment every number here came from (Python 3.9.25, PyTorch 2.8.0 with CUDA 12.8, MNE 1.8.0, scikit-learn 1.5.2) is listed in Table 18, and every pinned package in docs/act0_repro/requirements-act0.txt.

5.5 Impact Assessment

5.5.1 Assessment of Societal and Cultural Issues

Our TUAB result (Experiment 6) shows that a self-supervised encoder pre-trained only on a lower-resource-setting EEG dataset (FEI, pre-trained on NMT) can carry a measurable benefit to a different hospital's data without seeing any of its recordings in pre-training (the linear probe on top is still trained on that hospital's labels).

5.5.2 Assessment of Health and Safety Issues

FEI's false-alarm rate in young children (§5.2.1), and the other demographic-specific failure modes in §5.3.4, mean FEI+GTJ should stay a research artefact for now: not a clinical decision-support tool, not without age-stratified validation and recalibration first.

5.5.3 Assessment of Legal Issues

The official TUH Abnormal EEG Corpus is distributed by Temple University under a Data Use Agreement (DUA) signed with the corpus's maintainers (NEDC) before access is granted. The copy we used came from an unofficial, publicly re-hosted Kaggle mirror: it carries the corpus's own v3.0.1 README, and its directory layout and file counts (1,371 normal and 1,346 abnormal training recordings, 150 and 126 for evaluation) match that README's, but we never had a DUA-covered official copy to compare against, so we cannot show it is byte-for-byte identical to the licensed release (the SHA-256 of every file we used is in docs/act0_repro/raw_recordings_sha256.csv, Table 21). It is **not covered by that licence**. So every TUAB-derived number in this report (Experiments 1, 2, 3, 4, 6, and the visual check) is presented **for internal academic evaluation within this course submission only**, and should not be published, redistributed, or reused outside it. Any future public-facing use — a paper, a public leaderboard entry, a public repository result — would need either a properly licensed copy of TUAB through the official DUA process, or would need to drop the TUAB numbers and rely only on NMT and the openly-licensed stroke dataset.

5.6 Sustainability Evaluation

Small models and thermal-guarded, single-GPU training also limit compute demand (§5.2.2). CBraMod and LaBraM [5, 6] pre-train on tens of thousands of hours, while our two encoders have 1.15M (FEI) and 1.88M (GTJ) parameters, online and target copies included (Tables 1–2), and were trained for 60 epochs on a few thousand recordings. It still clears a classical baseline and, once TUAB's unlabelled recordings are added to pre-training, scores close to a released large-scale checkpoint's linear-probe score on TUAB, with no significant difference. If that holds more broadly, EEG models could be sized to the data available in a clinical setting, often small and expensive to collect, instead of defaulting to the biggest architecture on the shelf.

5.7 Ethical Issues

All three datasets were collected, de-identified, and released by their original curators under their own ethical review and consent processes; we are secondary users of already-public, de-identified data and collected none ourselves. We disclosed that our TUAB copy came from an unofficial source (§5.5.3), limited its numbers to this internal submission, and stated what proper licensing would require. A model with an undisclosed, demographic-specific failure mode (FEI over-flagging healthy young children at six times the adult rate) could do real harm if someone mistook it for a finished clinical tool instead of a term project.

6 Reflection on Individual and Team work

6.1 Individual Contribution of Each Member

Ahmed Mashfiqur Rahman (2106114) led the model architecture: studying V-JEPA and ViT-M, and designing and building both FEI and GTJ, including the attention-pooling and linear classifier heads and the shape/parameter checks that validated them.

Soumya Das Prottay (2106116) led data processing across all three datasets used in this report — NMT, TUAB, and the stroke cohort — including the shared preprocessing pipeline reproducing the paper's §4.1 recipe, the per-dataset windowing and training-subset construction, and the data-quality checks behind them, as well as the stroke spectrum-matching fix (Experiment 8) and the BSI feature extraction behind the clinical baseline (Experiment 7).

Maruf Hasan Mahi (2106121) led training and results: running the pretraining and fine-tuning jobs, the released-checkpoint reproduction and representation-collapse diagnosis (Experiments 1–4), the cross-validation and statistical-significance tests behind every reported number, the tables and figures, and the failure-axis analysis in §5.3.4.

Beyond these individual leads, all three team members took part in the joint design discussions and experiment-design decisions described under Mode of Teamwork below, and in reviewing every result before it was accepted into this report.

6.2 Mode of TeamWork

We made major design decisions together, including which failure mode to investigate next, which experiment a result needed before we trusted it, and what to drop from scope. Members then implemented their parts individually. We reviewed each result together before accepting it.

6.3 Diversity Statement of Team

Group 01 brought together interests in signal processing, machine learning, and clinical applications. That mix shaped the project: we treated EEG as a signal with its own structure, rather than a generic input to a black-box model.

6.4 Log Book of Project Implementation

Our proposal was accepted in week 5 of the term (Level 4, Term I). Work continued through week 7, followed by a one-week mid-term break, after which we resumed in week 8 and worked through to week 14, when this report and its accompanying demonstration were finalised.

Table 17: Log book of Act-0 project implementation, week 5 through week 14.

Week	Milestone achieved	Individual role	Team role
Week 5	Proposal accepted; literature review of EEG-VJEPA, the JEPA paradigm, and existing EEG foundation models	Reading, initial pipeline scoping	Joint proposal writing
Week 6	Hardware thermal-guard set up; NMT dataset acquired; paper's §4.1 preprocessing pipeline reproduced	Preprocessing implementation	Design discussion, review
Week 7	Released ViT-M/ViT-B checkpoints evaluated on NMT-matched input; representation collapse first suspected	Evaluation pipeline, diagnostics	Joint result review

Week	Milestone achieved	Individual role	Team role
Mid-term break	FEI designed and implemented from scratch; validated with unit self-tests	FEI implementation and testing	Design discussion
Week 8	GTJ designed and implemented; both branches unit-tested individually before any fusion	GTJ implementation and testing	Design discussion, review
Week 9	FEI+GTJ fusion built; 5-fold NMT cross-validation run against the band-power baseline (Exp. 5)	Fusion + CV pipeline	Joint result review
Week 10	TUAB dataset acquired and preprocessed; released-checkpoint and FEI+GTJ transfer evaluation begun (Exp. 1, 6)	TUAB pipeline, transfer evaluation	Joint result review
Week 11	Collapse formally diagnosed (Exp. 3); fine-tune-vs-scratch (Exp. 2) and simplest-probe (Exp. 4) experiments run; the bf16 fine-tuning benchmarks ran noticeably slower live than on paper once CPU thermal throttling kicked in on the longer jobs	Diagnostics, statistics	Design discussion, review
Week 12	Stroke dataset acquired; Brain Symmetry Index baseline implemented; pipeline-soundness gate passed (Exp. 7)	BSI implementation, gating	Joint result review
Week 13	GTJ's cross-dataset BatchNorm collapse on stroke diagnosed, traced to a silently wrong assumption that the stroke recordings used NMT's own filter band; spectrum-matching fix designed, gated, and applied (Exp. 8)	Fix design and implementation	Design discussion, review
Week 14	Failure-axis analysis completed; all results, figures and this report finalised for demonstration	Failure-axis analysis, report writing	Joint review and rehearsal

7 Communication to External Stakeholders

7.1 Executive Summary

A recently published AI model, EEG-VJEPa, claimed to automatically tell healthy brain recordings apart from abnormal ones by adapting a video-understanding technique to EEG. Our team tested this claim and found that the publicly released version does not actually work as described: it gives almost the same answer for every patient. We could show this directly. We built two smaller models instead: one for frequency content and one for how a signal's rhythm changes over time. Together they beat a simple classical measurement on the dataset

they were trained on (an untrained copy of the same design already does most of that) and score close to an established clinical measurement, with too few patients to tell the two apart, on a third task: telling stroke patients apart from healthy volunteers. On a different hospital's data, the frequency model's training still helped on its own, across five training runs. The rhythm model's training did not carry over, and retraining both models with that hospital's unlabelled recordings changed little: either way the combined model scored close to the released model and not measurably better than an untrained copy of itself.

7.2 Reproducing Our Results

All code, saved model checkpoints, and saved per-recording predictions used to produce every number, table, and figure in this report are organized as a set of scripts, one per experiment (Table 5). Table 4 gives the script behind each experiment's input. The 200 Hz NMT copy that FEI and GTJ read (per-channel z-scored at NMT's native rate, stored as 2.5-second frames) was first written in August 2026 by a script that was not kept; we reconstructed its recipe as `code/preprocess_nmt_200hz.py`, which reproduces all 2,417 stored files byte for byte. From those inputs, pre-train an encoder (`fei_pretrain.py` or `fei_branchC.py`, both with a `--selftest` flag for a fast sanity check with no real data and a `--cv` flag for the full five-fold cross-validation reported here), score it with `feic_nmt_anchor.py` (Experiment 5) or `tuab_feic.py` (Experiment 6), and evaluate a frozen or fine-tuned released checkpoint with the corresponding `tuab_*_eval.py` script. Every long-running script checkpoints its progress after each cross-validation fold and can be safely re-run to resume after an interruption, and each accepts command-line flags for every experiment variant reported here. The code is in github.com/mashfiqurrahman2020-coder/EEG-VJEPa-act-0, a private repository because it holds results computed on our unofficial TUAB copy (§5.5.3); course examiners can be given read access to the exact commit below. Its `docs/act0_repro/` folder holds the fold, subset and cohort lists, the SHA-256 of every checkpoint and result file, and the pinned environment (Appendix §11). The code, checkpoints and results behind this report are commit `d9cfacc0d36a19d7233b3a80792764c6c3bec6d0`, which unlike a tag cannot be moved: `git clone https://github.com/mashfiqurrahman2020-coder/EEG-VJEPa-act-0.git`, then `git checkout d9cfacc0d36a`, then `pip install -r docs/act0_repro/requirements-act0.txt` under Python 3.9. Appendix §11.6 gives the pre-training commands and §11.9 the data each experiment uses. The report itself is in a later commit, under the tag `act0-report-round15f`. All 70 FEI/GTJ checkpoints are committed in the repository (`code/*.pt`, about 180 MB), so nothing needs a separate download except the data and the released EEG-VJEPa weights. Two kinds of reproduction follow. The saved checkpoints and predictions reproduce every reported number exactly. Re-training does not: the original FEI and GTJ runs set no random seed, so they cannot be regenerated bit for bit, and cuDNN kernels were left non-deterministic, so even the seeded re-runs should be expected to reproduce the results statistically, not bit for bit.

8 Project Management and Cost Analysis

8.1 Computational Resources Used

This software project has no physical bill of materials. Compute was the resource budget. All pre-training and evaluation ran on a locally-owned workstation with one consumer-grade GPU (NVIDIA RTX 5060 Ti); we used no paid cloud compute. The direct monetary cost was the workstation's electricity. The practical cost that mattered day to day was **time and thermal headroom**: every script in this project is gated by a shared hardware guard that pauses training if CPU temperature rises above a safe threshold, which shaped how many experiments (and how many random-init control seeds) we could fit into the term.

To put a concrete number on it: `code/runs/act0/feic_joint.log` records the 5-fold cross-validation run that pre-trains FEI (1.15M parameters, Table 1) and GTJ (1.88M parameters, Table 2) — each independently, not against a shared loss — on the joint NMT+TUAB training pool, the "joint-pretraining" data variant compared against this report's main NMT-only numbers elsewhere. It trains 10 encoders in total (one FEI and one GTJ per fold), each for 60 epochs of 145 steps. Per-epoch time, sampled every 10th epoch across all 10 runs, ranged from 2 s to 24 s (FEI averaging ≈ 5.7 s/epoch; GTJ averaging ≈ 9.8 s/epoch, roughly double, reflecting its larger encoder and sequential GRU), with no systematic drift across epochs — the spread is scheduling noise from the shared GPU and the thermal guard, not a slowdown over time. Extrapolating each run's sampled mean to its full 60 epochs puts the entire 10-run job at approximately 78 minutes of GPU time; this is an estimate (only every 10th epoch is timestamped in the log), not an exact sum.

8.2 Timeline of Project Implementation

See Table 17 (§6.4) for the detailed week-by-week log book. At a glance: weeks 5–7 covered proposal, literature review, data acquisition, and the first (unsuccessful) reproduction attempt; the mid-term break and week 8 covered building FEI and GTJ from scratch, each unit-tested independently; weeks 9–11 covered NMT validation, TUAB transfer, and formally diagnosing representation collapse; weeks 12–13 covered the stroke cohort, the clinical BSI baseline, and the spectrum-matching fix; week 14 covered the failure-axis analysis and this report.

9 Future Work

- **Age-aware modelling.** FEI's false-alarm rate in healthy young children is far higher than in adults (§5.3.4), the clearest practical finding in this report. An age-conditioned probe, and AUROC reported by age band as standard practice, should follow.
- **A general fix for GTJ.** The spectrum-matching fix in Experiment 8 was built for the stroke cohort specifically; a dataset-agnostic version (replacing BatchNorm with per-recording normalisation, say) could let GTJ transfer the way FEI already does. The same work should cover GTJ's confidently-wrong errors (§5.3.4) too: a calibration step, such as temperature scaling fit on a held-out fold, before its output is read as a probability rather than a ranking. Before any of that, the spectral-matching recipe should be frozen and applied unchanged to a new stroke cohort: Experiment 8 only shows it repairing the cohort it was designed on.
- **A properly licensed TUAB evaluation.** Obtaining the official, DUA-covered TUH Abnormal EEG Corpus would let the TUAB-derived findings here be verified, extended, and published without the internal-only restriction in §5.5.3.
- **Motor imagery.** The team has already started a separate extension of this work into motor-imagery brain-computer interfaces, building on GTJ's spectral-dynamics design, out of scope for this report.

10 References

- [1] Hojjati, A., Li, L., Hameed, I., Yazidi, A., Lind, P. G., & Khadka, R. (2025). From Video to EEG: Adapting Joint Embedding Predictive Architecture to Uncover Spatiotemporal Dynamics in Brain Signal Analysis. arXiv:2507.03633. <https://arxiv.org/abs/2507.03633>
- [2] Assran, M., Duval, Q., Misra, I., Bojanowski, P., Vincent, P., Rabbat, M., LeCun, Y., & Ballas, N. (2023). Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture. arXiv:2301.08243 (CVPR 2023). <https://arxiv.org/abs/2301.08243>
- [3] Bardes, A., Garrido, Q., Ponce, J., Chen, X., Rabbat, M., LeCun, Y., Assran, M., & Ballas, N. (2024). Revisiting Feature Prediction for Learning Visual Representations from Video. arXiv:2404.08471. <https://arxiv.org/abs/2404.08471>
- [4] Grill, J.-B., Strub, F., Altché, F., Tallec, C., Richemond, P., et al. (2020). Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning. arXiv:2006.07733 (NeurIPS 2020). <https://arxiv.org/abs/2006.07733>
- [5] Wang, J., Zhao, S., Luo, Z., Zhou, Y., Jiang, H., Li, S., Li, T., & Pan, G. (2025). CBraMod: A Criss-Cross Brain Foundation Model for EEG Decoding. arXiv:2412.07236 (ICLR 2025). <https://arxiv.org/abs/2412.07236>
- [6] Jiang, W.-B., Zhao, L.-M., & Lu, B.-L. (2024). Large Brain Model for Learning Generic Representations with Tremendous EEG Data in BCI. arXiv:2405.18765 (ICLR 2024, Spotlight). <https://arxiv.org/abs/2405.18765>

- [7] Khan, H. A., Ul Ain, R., Kamboh, A. M., Butt, H. T., Shafait, S., Alamgir, W., Stricker, D., & Shafait, F. (2022). The NMT Scalp EEG Dataset: An Open-Source Annotated Dataset of Healthy and Pathological EEG Recordings for Predictive Modeling. *Frontiers in Neuroscience*. doi:10.3389/fnins.2021.755817. <https://doi.org/10.3389/fnins.2021.755817>
- [8] López, S., Suarez, G., Jungreis, D., Obeid, I., & Picone, J. (2015). Automated Identification of Abnormal Adult EEGs. *IEEE Signal Processing in Medicine and Biology Symposium (SPMB)*. <https://pmc.ncbi.nlm.nih.gov/articles/PMC4868184/>
- [9] van Putten, M. J. A. M., & Tavy, D. L. J. (2004). A brain symmetry index (BSI) for online EEG monitoring in carotid endarterectomy. *Clinical Neurophysiology*, 115(5), 1189–1194. <https://doi.org/10.1016/j.clinph.2003.12.002>
- [10] van Putten, M. J. A. M. (2007). The revised brain symmetry index. *Clinical Neurophysiology*, 118(11), 2362–2367. <https://doi.org/10.1016/j.clinph.2007.07.019>
- [11] Zare, M. (2026). A Negative-Control Protocol for Clinical EEG Foundation-Model Benchmarks: Dataset Identity and External-Cohort Stress Testing. arXiv:2607.24519 (preprint). <https://arxiv.org/abs/2607.24519>
- [12] Orvieto, A., Smith, S. L., Gu, A., Fernando, A., Gulcehre, C., Pascanu, R., & De, S. (2023). Resurrecting Recurrent Neural Networks for Long Sequences. arXiv:2303.06349 (ICML 2023). <https://arxiv.org/abs/2303.06349>
- [13] Zenodo record 19599466. Resting-state EEG recordings, healthy controls and stroke patients (used for the healthy-vs-stroke classification task in this report). <https://zenodo.org/records/19599466>
- [14] Zhang, X., Zhao, Z., Tsiligkaridis, T., & Zitnik, M. (2022). Self-Supervised Contrastive Pre-Training For Time Series via Time-Frequency Consistency. arXiv:2206.08496 (NeurIPS 2022). <https://arxiv.org/abs/2206.08496>
- [15] van den Oord, A., Li, Y., & Vinyals, O. (2018). Representation Learning with Contrastive Predictive Coding. arXiv:1807.03748. <https://arxiv.org/abs/1807.03748>
- [16] Chung, Y.-A., Hsu, W.-N., Tang, H., & Glass, J. (2019). An Unsupervised Autoregressive Model for Speech Representation Learning. arXiv:1904.03240 (Interspeech 2019). <https://arxiv.org/abs/1904.03240>
- [17] Banville, H., Chehab, O., Hyvärinen, A., Engemann, D.-A., & Gramfort, A. (2020). Uncovering the structure of clinical EEG signals with self-supervised learning. arXiv:2007.16104. <https://arxiv.org/abs/2007.16104>
- [18] Zhou, J., Cao, Z., Duan, Y., Barkley, C., Leong, D., Jiang, X., Nguyen, Q.-T., Zhao, Z., Do, T., Chang, Y.-C., Liang, S.-F., & Lin, C.-T. (2025). Pretraining Large Brain Language Model for Active BCI: Silent Speech. arXiv:2504.21214. <https://arxiv.org/abs/2504.21214>
- [19] Sun, G., Ma, J., Wu, Y., Tang, W., & Miao, Y. (2026). EEG-VID: Task-Guided Latent Predictive Pretraining for EEG Decoding and Assistive Target Selection. arXiv:2609.00566 (preprint). <https://arxiv.org/abs/2609.00566>
- [20] Nadeau, C., & Bengio, Y. (2003). Inference for the Generalization Error. *Machine Learning*, 52(3), 239–281. <https://doi.org/10.1023/A:1024068626366>
- [21] Holm, S. (1979). A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics*, 6(2), 65–70. <https://www.jstor.org/stable/4615733>
- [22] Sheorajpanday, R. V. A., Nagels, G., Weeren, A. J., van Putten, M. J. A. M., & De Deyn, P. P. (2009). Reproducibility and clinical relevance of quantitative EEG parameters in cerebral ischemia: A basic approach. *Clinical Neurophysiology*, 120(5), 845–855. doi:10.1016/j.clinph.2009.02.171. <https://doi.org/10.1016/j.clinph.2009.02.171>

11 Appendix: Reproducibility Records

11.1 Environment and Checkpoint Provenance

Table 18: Software environment, upstream code and released-checkpoint provenance (docs/act0_repro/environment.txt and checkpoints_sha256.csv).

Item	Value
Python	3.9.25
PyTorch	2.8.0+cu128 (CUDA 12.8, cuDNN 9.10.2)
Key packages	MNE 1.8.0, scikit-learn 1.5.2, SciPy 1.13.1, NumPy 1.26.4, pandas 2.3.3 (all 111 pins in docs/act0_repro/requirements-act0.txt)
GPU	NVIDIA GeForce RTX 5060 Ti, driver 595.91.07
Operating system	Linux 7.0.0-38-generic, x86-64, glibc 2.39
Upstream EEG-VJEPa code	github.com/amir-hojjati/eeg-vjepa, commit c739fad5ad57ec4439a094343901a245edf611ec
Released checkpoints	Hugging Face amir-hlp/EEG-VJEPa, commit ec98405f5f3423fd59d3296157ee1570efd8d947
ViT-M (4×30×4) SHA-256	5fee4a6c08123cfb2d18d86e5d483aac32630ceed7ed1b2babc052799c48db79
ViT-B (4×30×2) SHA-256	66e9673f8ce42c3c973470821777a793a37db2ddb58dfd040a664560f7eefc85
Our 70 FEI/GTJ checkpoints	size and SHA-256 of each in docs/act0_repro/checkpoints_sha256.csv
Our 38 result files	size and SHA-256 of each in docs/act0_repro/results_sha256.csv

11.2 Stroke Cohort: Pre-registration, Post-hoc Input and Fold-local Check

The stroke analysis was pre-registered in a local file, runs/act0/phaseB/PREREGISTRATION.md, written on 2026-09-25 at 23:55 (UTC+6) before any LOSO run. stroke_phaseB.py refuses to run its LOSO stage without that file and stores its SHA-256 (c66463066a4f3dc50e21cf82f3debc2c5d5b1ae4d8ac6ec1502e9617ba8d7321) in every result. It is a timestamped file in our own working copy, not an entry in a public registry, so it records our order of work without proving it to an outsider. Under the pre-registered input, FEI+GTJ scored 0.833 LOSO AUROC, against 0.889 for its random-init twin and 0.907 for BSI. The spectral-matching input of Experiment 8 was designed after that result: its recipe, gates and comparisons were written to runs/act0/phaseB_clean/POSTHOC.md on 2026-09-26 at 19:06 (SHA-256 f78446790f0e7d19e6d87f3c1ffd6265fe42f1d3e50612d5a4a84f3f588529c8) before any LOSO run on that input, so every Experiment 8 number is post-hoc in that sense. Its filter is fitted once, without labels, on all 15 subjects pooled, so each held-out subject helps shape the filter applied to it. To check that this does not drive the result, stroke_phaseB_foldlocal.py refits the filter inside every LOSO fold on the 14 training subjects only and applies it unchanged to the held-out one. Table 19 compares the two. The pre-registration also fixes both cohorts. The primary cohort takes, for every subject, the same 285 s of pure rest before any task marker (the shortest rest block, PAC05's, is 285.0 s): stroke PAC02–PAC10 (9) against the dataset authors' own six controls. PAC01 and controls 02 and C01 are left out because they have at most 52 s of rest before the task. The sensitivity cohort follows the authors' definition, the first 300 s of every file (10 stroke, 8 controls); for PAC01, 02 and C01 that window includes task data, so it is reported as a sensitivity check only.

Table 19: Stroke primary cohort, LOSO AUROC with the spectral-matching filter fitted on all 15 subjects pooled (Experiment 8) or refitted inside each fold on the 14 training subjects. BSI alone (0.907) uses no filter.

Arm	Pooled filter (Experiment 8)	Fold-local filter
FEI+GTJ	0.926	0.926
FEI+GTJ+BSI	0.963	0.944
rand-FEI+GTJ	0.944	0.944
rand-FEI+GTJ+BSI	0.944	0.944
FEI	1.000	1.000

Arm	Pooled filter (Experiment 8)	Fold-local filter
GTJ	0.833	0.833
rand-FEI	0.907	0.907
rand-GTJ	0.852	0.852

Only the combined FEI+GTJ+BSI arm moves (0.963 to 0.944), which removes its small numerical edge over its random-init twin; FEI+GTJ itself, and its AUROC close to BSI's that Experiment 8 rests on, are unchanged.

11.3 Experiment 5 Corrected Statistics

Table 20: Experiment 5 on NMT, all eleven comparisons. Pre-trained arms are per-fold means over three pre-training runs, random-init arms over five seeds. Corrected p is the Nadeau–Bengio resampled t-test [20] (variance $\times (1/5 + n_{\text{test}}/n_{\text{train}})$, $n_{\text{train}} \approx 1,934$, $n_{\text{test}} \approx 483$, 4 degrees of freedom); Holm [21] adjusts over all eleven. Naive p is the ordinary paired t-test over folds, shown for comparison only. Averaging the three runs within each fold before the test folds training-run variation into the fold means; a hierarchical model would estimate the two sources separately.

A vs B	Mean A	Mean B	Δ AUROC	Folds A > B	Naive p	Corrected p	Holm p
FEI vs band power	0.820	0.786	+0.034	4/5	0.038	0.112	0.447
GTJ vs band power	0.831	0.786	+0.045	5/5	< 0.001	0.003	0.027
FEI+GTJ vs band power	0.861	0.786	+0.075	5/5	< 0.001	0.004	0.029
rand-FEI vs band power	0.810	0.786	+0.024	4/5	0.109	0.243	0.486
rand-GTJ vs band power	0.794	0.786	+0.009	5/5	0.027	0.086	0.431
rand-FEI+GTJ vs band power	0.834	0.786	+0.048	5/5	< 0.001	0.002	0.026
FEI vs rand-FEI	0.820	0.810	+0.010	3/5	0.179	0.338	0.486
GTJ vs rand-GTJ	0.831	0.794	+0.036	5/5	0.005	0.019	0.116
FEI+GTJ vs rand-FEI+GTJ	0.861	0.834	+0.027	5/5	0.055	0.149	0.447
FEI+GTJ vs FEI	0.861	0.820	+0.041	5/5	< 0.001	0.003	0.027
FEI+GTJ vs GTJ	0.861	0.831	+0.030	5/5	0.003	0.012	0.082

11.4 Reproducibility Files

Table 21: Files in docs/act0_repro/ (written by code/act0_repro_manifest.py). The raw recordings are not redistributed: NMT [7] and the stroke cohort [13] are public, and TUAB must be obtained from its maintainers under their data use agreement (§5.5.3). Their per-file SHA-256 are listed instead, so any copy can be checked byte for byte.

File	Rows	Contents
nmt_folds_seed0.csv	2,417	Every NMT recording's label and test fold (Experiment 5)
tuab_subsets.csv	2,730	The five 546-recording TUAB training subsets s0–s4: file and subject
stroke_cohorts.csv	33	Primary (15) and sensitivity (18) cohort membership, label and rest seconds
checkpoints_sha256.csv	72	Size and SHA-256 of the released checkpoints and every FEI/GTJ checkpoint
results_sha256.csv	38	Size and SHA-256 of every saved result file under code/runs/act0
requirements-act0.txt	111	Pinned Python packages
environment.txt	6	Table 18's environment lines
raw_recordings_sha256.csv	5,501	Dataset, path, size and SHA-256 of every file in the three raw-data folders: TUAB's 2,993 EDFs; NMT's 2,417 EDFs and its label file; the stroke folder's 90 files (27 BrainVision recordings plus the archive's other files and our download script and log) (code/act0_raw_hashes.py)

11.5 Checks Not Run

Table 22: Checks that would strengthen this report but were not run, why, and what the report gives in their place.

Check	Why it was not run	What the report gives instead
Re-running EEG-VJEP A pre-training with reg_coeff restored (Experiment 3)	The released config (cluster_vit16_EEG.yaml) pre-trains for 400 epochs. One run of that length, let alone the repeats a causal test needs, did not fit the GPU time we had.	The collapse itself is measured (Table 9); reg_coeff = 0 and the lowest-loss checkpoint (epoch 353) are given as hypotheses for its cause.
Nested cross-validation for GTJ's window length (Experiment 5)	The 5/20/40 s comparison would have to be repeated inside every outer fold, multiplying GTJ pre-training by the number of inner folds.	The selection is disclosed with the sweep, and GTJ beats band power at every window length (0.820, 0.836, 0.823).
NMT-only GTJ and FEI+GTJ re-runs on TUAB (Experiment 6)	The TUAB re-runs went to FEI, whose NMT-only gain is the claim the report makes; NMT-only GTJ already scores below its random-init twin, and no conclusion rests on it.	NMT-only GTJ and FEI+GTJ from the original run against their twins (Table 12); FEI over five runs and the adapted encoders over three (§5.3.3).
Permutation tests for Experiment 8's pre-training re-runs	Each needs 1,000 full LOSO re-runs per arm and run, the slowest part of Experiment 8; the re-runs were added in the last revision.	Paired bootstrap comparisons of each re-run with the same random-init twin and with BSI over the 15 subjects (§5.3.3).
Five random-init seeds for the TUAB and stroke twins	Not run in the time left before submission; the three-seed twin ensemble is the reference behind every reported TUAB and stroke pre-training gain.	Experiment 6's multi-run bootstrap resamples the three seeds as a level of its own (Table 24), so their variation enters the interval, if coarsely.
Repeated, deterministic runs of the Fig-3 attentive head (Experiment 1)	No conclusion depends on that head's exact value; the argument uses the linear probe and the §4.2 head, which is bit-identical across re-runs.	Both runs are reported (72.1 and 68.7) and Table 7 uses the re-run.
Evaluation on the licensed TUAB release	Access needs a data use agreement with TUAB's maintainers (§5.5.3), which we did not have.	TUAB numbers are restricted to this submission (§5.5.3); a licensed evaluation is listed as future work.
A container image (e.g. Docker)	Not built in the time available.	docs/act0_repro/ pins all 111 Python packages and records Python, PyTorch, CUDA, cuDNN, GPU driver and OS (Table 18).
Saved embeddings (emb_*.npz, about 446 MB)	Too large for the repository; each is a forward pass of a published checkpoint or of a seeded random-init twin.	The repository holds the per-recording predictions (preds_*.npz, oof_*.npz, kf5_lpo_*.npz, 7.3 MB) behind every reported score, plus the checkpoints that regenerate the embeddings.
An equivalence test between FEI+GTJ and BSI on stroke (Experiment 8)	No equivalence margin was set in advance, and with 15 subjects the 95% CI of the difference (−0.240 to 0.280) is far wider than any useful margin.	The two are reported as not separable, not as equivalent.
The spectral-matching recipe, frozen, on a new stroke cohort (Experiment 8)	We had no second stroke EEG cohort within the project.	Experiment 8 is reported as a post-hoc repair on the cohort that prompted it; external validation is future work.
An archived release with a DOI	The repository is private because it holds results from our unofficial TUAB copy (§5.5.3), and a DOI archive (Zenodo, for example) is public.	§7.2 gives the full 40-character commit SHA, which fixes the exact code, checkpoints and results; course examiners can be given read access to it.

11.6 Pre-training Configuration and Commands

Every encoder pre-trained for this report uses the recipe below. None of the runs uses a learning-rate schedule, mixed precision, gradient clipping or PyTorch's deterministic-algorithm flags, and each keeps its final (60th) epoch's weights, with no checkpoint selection. cuDNN kernels were left non-deterministic, so even the seeded runs are not bit-identical on a re-run. Random-init twins are built from the same architectures with torch seed = twin index (seeds 0–2 on TUAB and stroke, 0–4 on NMT); Experiment 6's multi-run bootstrap resamples the three TUAB seeds (Table 24).

Table 23: Pre-training data, seeds, recipe and command for every encoder trained in this report. All share AdamW (learning rate 2×10^{-4} , weight decay 10^{-4}), batch 128, 60 epochs, 4 random windows per recording per epoch, $d = 256$, $h = 128$ and target EMA 0.996.

Encoders	Pre-training data and seeds	Window and branch settings	Command and checkpoints
FEI, NMT only (Experiments 5, 6, 8)	The NMT training folds of the seed-0 5-fold split (Table 21); no seed set (the original runs are unseeded)	$L = 1,000$ samples (5 s at 200 Hz)	<code>python code/fei_pretrain.py --cv → fei_enc_cv_s0_f{k}.pt</code>
GTJ, NMT only (Experiments 5, 8)	As for FEI; no seed set	$L = 4,000$ (20 s), STFT n_fft 200, hop 100, context 60%, GRU	<code>python code/fei_branchC.py --cv --L 4000 --hop 100 --tag h20 → branchC_h20_enc_cv_s0_f{k}.pt</code>
FEI and GTJ replicates 1 and 2 (Experiment 5, Table 20; FEI also in Experiment 6, both in Experiment 8)	As above; torch, NumPy and Python random seeded with $100 \times \text{replicate} + \text{fold}$	As above	<code>python code/act0_nmt_pretrain_replicate.py</code>
FEI replicates 3 and 4 (Experiment 6 only)	As for replicates 1 and 2	As for FEI	<code>python code/tuab_fei_reps.py → fei_rep{3,4}_enc_cv_s0_f{k}.pt</code>
FEI and GTJ, joint NMT+TUAB (Experiment 6)	NMT training fold k plus TUAB's 2,717 unlabelled training recordings; torch seed = k	As above	<code>python code/feic_joint.py → fei_joint_enc_cv_s0_f{k}.pt, branchC_h20_joint_enc_cv_s0_f{k}.pt</code>
FEI and GTJ, joint NMT+TUAB, replicates 1 and 2 (Experiment 6)	As for the joint encoders; torch, NumPy and Python random seeded with $100 \times \text{replicate} + \text{fold}$	As above	<code>python code/tuab_feic_joint_reps.py → fei_joint_rep{r}_enc_cv_s0_f{k}.pt, branchC_h20_joint_rep{r}_enc_cv_s0_f{k}.pt</code>

11.7 Statistical Procedures

Table 24: The statistical test behind each kind of comparison in this report.

Comparison	Procedure	Where used
Confidence interval of one model's AUROC or BAcc	Percentile bootstrap over test recordings (subjects on stroke), 2,000 resamples, seed 0, 2.5th–97.5th percentiles (code/act0_metrics.py)	Experiments 4, 6, 8
Two models on the same test set (Δ AUROC)	Paired bootstrap: both models scored on the same 2,000 resamples; CI from the percentiles of Δ , and p = share of resamples with $\Delta \leq 0$ (one-sided)	Experiments 6 and 8 (e.g. FEI+GTJ – ViT-M on TUAB, $p = 0.147$)

Comparison	Procedure	Where used
Mean pre-training gain over several pre-training runs (Δ AUROC)	Multilevel bootstrap, 2,000 resamples, seed 0: each resample draws, with replacement, the pre-training runs, the random-init twin's three seeds (re-ensembled) and the test recordings; Δ = mean AUROC of the drawn runs minus the drawn twin ensemble's AUROC; CI from the percentiles of Δ , p = share with $\Delta \leq 0$ (pooled() in code/tuab_fei_reps.py). Against the released ViT-M, one fixed model, only runs and recordings are drawn.	Experiment 6 (five NMT-only FEI runs; three adapted runs)
Pre-training re-runs on the stroke cohort	Each run scored on its own with the paired bootstrap over the same 15 subjects; runs are not pooled, so their spread (seed variation) stays separate from subject-level uncertainty.	Experiment 8
Released vs random-weight ViT-M	One-sided exact Mann–Whitney U on per-run final AUROCs: 5 released-checkpoint runs (one head per training subset, s0–s4) against 3 random-weight runs (s0–s2, each with its own random weights, torch seed = subset number). Alternative: released below random in Experiment 1, above random in Experiment 4. With 5 and 3 runs the smallest possible p is $1/56 = 0.018$; two-sided p -values are twice the one-sided ones.	Experiments 1 and 4
Fine-tuned vs from-scratch ViT-M on the same five training subsets	Two-sided paired t-test and Wilcoxon signed-rank over the 5 subsets. Any two subsets share 98–134 of their 546 recordings, which both tests ignore, so their p -values lean small; at $p = 0.81$ that cannot change the conclusion.	Experiment 2
Two arms on the same NMT folds	Paired t-test over the 5 folds; Appendix §11.3 repeats all eleven with the Nadeau–Bengio corrected t-test and Holm's adjustment	Experiment 5 (Table 20)
Stroke: one arm against chance	Label permutation, the full LOSO re-run for each of 1,000 shuffles; $p = (1 + \#\text{null} \geq \text{observed}) / 1,001$	Experiments 7 and 8
Stroke: robustness to the split	100 repeats of stratified 5-fold with the same splits for every arm (win rate = share of repeats arm A beats arm B; permutation p from 200 shuffles $\times$ 10 repeats); leave-pair-out with a stratified 2,000-resample bootstrap over subjects	Experiment 8 (Table 15, Figure 19)
Stroke: classical features, stroke vs control	Two-sided Mann–Whitney U; the pre-declared gate is broadband BSI at $p < 0.05$	Experiment 7 (Table 13)

Only Experiment 5's eleven NMT comparisons are adjusted for multiplicity (Holm, Table 20). The stroke pre-registration fixed the gate and three paired comparisons, all involving the released ViT-M (ViT-M+BSI vs BSI, ViT-M vs its random twin, FEI+GTJ vs ViT-M) on the pre-registered input; Experiment 8's comparisons on the spectrum-matched input were fixed in POSTHOC.md before any LOSO run on that input, but after the pre-registration. Every other p -value in this report is exploratory and unadjusted: evidence about that one comparison, not a claim about the whole family.

11.8 Stroke Spectral-Matching Filter

Experiment 8's input is built by code/stroke_phaseB_clean.py, following runs/act0/phaseB_clean/POSTHOC.md, which was written before any LOSO run on this input. Each recording gets the dataset authors' bad-channel interpolation, a common average reference, the 19 NMT channels (T7/T8/P7/P8 renamed T3/T4/T5/T6), resampling to 200 Hz with no band-pass, and a per-channel z-score. The

target is NMT's mean log10 Welch spectrum over 100 fixed-random NMT training recordings (first 300 s; 800-sample Welch segments, i.e. 0.25 Hz bins; 10^{-20} added before the log).

The filter is a log10 amplitude gain per channel on that 0.25 Hz grid. It starts at zero and is updated four times: shape every subject, z-score again, average the cohort's log-spectra, and add half the remaining gap to NMT's. Shaping multiplies the recording's FFT by the gain, linearly interpolated onto the FFT's frequency grid with no smoothing; the gain is real, so the filter changes no phase (zero-phase). Each recording is reflect-padded by 800 samples at both ends before the FFT and trimmed after it. The result is z-scored again and cut into non-overlapping 2.5 s frames.

One filter per channel is fitted on all 15 subjects pooled, without labels, and the same filter is applied to every subject (Experiment 8); Table 19 refits it inside each LOSO fold on the 14 training subjects. Four gates were fixed in POSTHOC.md and reported whether or not they passed: the matched cohort's mean log-spectrum within 0.25 decades of NMT's at every channel and bin from 0.5 to 99 Hz; GTJ's input BatchNorm |z| at most 1.5 in every 1 Hz bin, against each NMT checkpoint's running statistics; GTJ's across-subject embedding spread at least half its spread on 15 NMT evaluation recordings; and at most 5% of its embedding dimensions constant ($\text{std} < 10^{-5}$). All four passed (§5.3.3).

11.9 Data Used by Each Experiment

Table 25: Which recordings each experiment's encoder was pre-trained on, which trained its classifier, and which scored it. No evaluation recording is used for pre-training or probe training.

Experiment	Encoder pre-training	Classifier training	Evaluation
1, 2, 4	Released ViT-M: the authors' own pre-training; random-weight twins: none	TUAB, the five 546-recording training subsets and (Experiment 4) the full 2,717-recording training set; Experiment 2 fine-tunes the whole network on the same subsets	TUAB official eval set (276)
3	Released ViT-M and its random-weight twins	None (no classifier)	TUAB recordings and the 15-subject stroke cohort (Table 9)
5	FEI, GTJ: the NMT training folds of the seed-0 5-fold split ($\approx 1,934$ recordings), labels unused	The same NMT training fold, with labels	The held-out NMT fold (≈ 483 recordings)
6, FEI row	The NMT-only FEI fold encoders of Experiment 5's three pre-training runs, plus two further FEI-only runs	As Experiment 4 (TUAB training subsets)	TUAB official eval set (276)
6, GTJ and FEI+GTJ rows	NMT training fold plus TUAB's 2,717 unlabelled training recordings (three pre-training runs)	As Experiment 4	TUAB official eval set (276)
7	None (BSI and spectral features)	LOSO: the other 14 stroke-cohort subjects	The held-out subject, all 15 in turn
8	The NMT-only fold encoders of Experiment 5 (all three pre-training runs); released ViT-M	LOSO: the other 14 subjects, after the spectral-matching filter (§11.8)	The held-out subject; also 100× repeated 5-fold and leave-pair-out